The present work was submitted to Lehr- und Forschungsgebiet Educational Technologies at DIPF

Development of a Library for Automatic Pseudonymization

Bachelor-Thesis

Presented by

Popovych, Oleksandra

7377543

First examiner: Prof. Dr. Hendrik Drachsler

Supervisor: Dr. Daniel Schiffner

Frankfurt, June 25, 2024

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich

Popovych, Oleksandra
(Nachname, Vorname)

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde.

Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den

25.06.2024,

Unterschrift der/des Studierenden

Abstract

Data pseudonymization has gradually become an essential part of data protection for all relevant parties that store, process, and use personal data. The measures are taken not only from the moral perspective of protecting user or customer data but also from the legal perspective of the General Data Protection Regulation. Although there are many anonymization and pseudonymization options available, the goal of the automated pseudonymization library was to create a general tool that is free of charge, allows reversible de-identification of data, is relatively easy to configure, and does not require the involvement of third parties in data processing.

This thesis describes the implementation of the pseudonymization library, discusses the literature research on current pseudonymization practices, outlines the technologies used for the development, and provides test results on the library's performance and functionalities. In addition, a graphical user interface is presented as an exemplary use case for a library and a simplified way to achieve pseudonymization for users without prior programming knowledge.

Contents

1 Introduction **1**

2 Background and related work **3**

2.1 Definition and Challenges of Pseudonymization 3

2.1.1 Sensitive Data 3

2.1.2 The Definition of Pseudonymization 4

2.1.3 The Concept of Reversible Pseudonymity 5

2.1.4 Pseudonymization of Unstructured Text 7

2.2 Pseudonymization Methods 8

2.2.1 Counter . . . 9
2.2.2 Random number generator (RNG) . . . 9
2.2.3 Cryptographic hash function . . . 10
2.2.4 Cryptographic encryption . . . 10
2.2.5 Merkle Tree . . . 11
2.2.6 Aggregation . . . 12
2.2.7 k-Anonymity . . . 12

3 Concept **15**
3.1 Goals, Purpose and Target Audience . . . 15
3.2 Key Features and Functionality . . . 16
3.3 Brainstorming Process . . . 16
3.4 GUI Concept . . . 18

4 Implementation **21**
4.1 About the Library . . . 21
4.2 Main Technologies . . . 23
4.3 Program Flow . . . 25
4.3.1 Pseudonymization . . . 25
4.3.2 Aggregation and k-Anonymization . . . 27
4.3.3 Reversibility . . . 28
4.3.4 Pseudonymization Methods . . . 28
4.3.5 GUI . . . 30
4.4 Use Cases and Scenarios . . . 32
4.4.1 Overview of Scenarios . . . 32
4.4.2 Scenario 1: Internal Data Storage . . . 33
4.4.3 Scenario 2: External Data Exchange and Disclosure of Information . . . 34
4.4.4 Scenario 3: Storage of Confidential Meeting Reports . . . 35
4.4.5 Scenario 4: Data Analysis and Research . . . 36
4.5 Documentation . . . 37

5 Testing **41**
5.1 Functionality Testing . . . 41
5.2 Library Performance Testing . . . 42
5.3 k-Anonymity Testing . . . 44
5.4 Pandas vs. Polars Performance Testing . . . 45
5.5 GUI Evaluation Survey . . . 46

6 Discussion **49**

7 Conclusion **53**

List of Tables and Figures **55**

Bibliography **57**

Appendices **61**

Appendix 1: Library Documentation **63**

Appendix 2: GUI Evaluation Form and Results **69**

1 Introduction

The General Data Protection Regulation[1] defines pseudonymization as a data protection measure that involves processing personal data so that the person cannot be directly identified without any additional information. Further, the additional information must be stored separately from other sensitive data, and the responsible parties must ensure its security. [20]

Several papers have recently been published on the possibility of re-identifying a person based on attributes that contain neither a name nor a direct identifier. The study about estimating the success of re-identifications in incomplete datasets [24] proposes that by utilizing 15 demographic attributes, around 99.98% of US citizens would be re-identified accurately within any dataset. Statista [28] reports that in 2023, there were around 3.205 cases of data breaches in the United States alone, affecting over 353 million people. Data pseudonymization reduces the risks, as the data is divided into pseudonymized data and additional information stored in separate databases. If the pseudonymized data is leaked, the attacker still cannot reveal the identity because additional information is necessary. Therefore, data pseudonymization and minimalization must be addressed and deployed as state-of-the-art for current and future developments.

Data pseudonymization has gradually become a widely discussed topic, primarily since the new GDPR was published in 2018.[20] The regulation defines pseudonymization and measures for storing and processing personal data by third parties. Consequently, the European Cybersecurity Agency[2] has contributed to the area by publishing technical reports on deploying pseudonymization techniques for various use cases, focusing on advanced techniques in the medical field. Numerous software solutions allow one to pseudonymize, link, and retain sensitive information. Medical institutions use solutions such as the ORCHESTRA Pseudonymization Tool to manage sensitive biomedical data. Organizations like Orion Innovations or Doccape(Skitlab) offer billable pseudonymization solutions for other companies. However, there

[1] GDPR
[2] ENISA

are nearly no standard open-source tools that focus specifically on pseudonymization. Most open-source tools anonymize data without data linking, thus not giving the possibility to re-identify data after the research is completed. Sometimes it is also safer if the data controller performs the pseudonymization directly on the personal computer without involving third parties in the data processing.

ENISA's recommendations [6] provide a comprehensive overview of the pseudonymization options available to the responsible party, taking into account potential solutions for data re-identification. However, implementing these recommendations is not a straightforward task. It requires starting from scratch, understanding the complexity of individual pseudonymization methods, developing new solutions and algorithms for re-identification, and conducting extensive testing.

This bachelor thesis focuses on developing a Python library for automatic data pseudonymization and describing current pseudonymization practices. The library allows anyone to apply pseudonymization to data even in its early development stages, thus complying with GDPR and other data security regulations. It enables its users to pseudonymize or anonymize data efficiently and effectively with the help of various predefined functions, thereby enhancing data security. The additionally developed graphical user interface makes pseudonymization accessible to an audience with no prior programming knowledge.

2 Background and related work

This chapter presents the basic definitions and concepts to help readers understand library implementation. The first part concentrates on the definition and challenges of pseudonymization, while the second part focuses on the pseudonymization methods applied in the library.

2.1 Definition and Challenges of Pseudonymization

2.1.1 Sensitive Data

According to the GDPR[1], personal data refers to any information that can help to directly or indirectly identify a natural person. This includes a name, ID number, and other possible demographic data, such as gender, location, and cultural or social identities. [20] Pseudonymization, especially under the GDPR, is often associated with protecting PII[2]. However, as Neerbek points out[18], the scope of sensitive data goes beyond personal or individual data. It also includes organizational or company data, where the risk of information leakage, such as inventions and strategies, is also a concern.

In addition, data protection is constantly evolving, making the definition of what constitutes sensitive data more complex and non-obvious. For example, a user surfing the internet may unintentionally disclose some of their data by sharing their IP address and browsing history with the service provider, which then serve as perfect attributes for re-identification.[3]

The data controller establishes the aims and reasons behind the processing of specific personal data. In contrast, the data processor is responsible for the actual processing of personal data, which must be supervised by the data controller. Both descriptions can be applied to one single person or authority. [6] This thesis divides sensitive data into structured and unstructured data while considering different scopes of data use cases, e.g., personal or organizational

[1] General Data Protection Regulation
[2] Personal Identifiable Information

data. Also, the data controller and the data processor could be referred to in some of the next chapters.

2.1.2 The Definition of Pseudonymization

The term *pseudonym*, used in English, is derived from the Greek word *pseudōnymos*, meaning *false name*. Therefore, pseudonym can be described as a *false* name that replaces the actual name and can be appended to a concrete single person. This is why most data privacy regulations often handle pseudonyms as sensitive data, as they still have the potential to reveal a person's actual identity. [3]

The initial definition of data pseudonymization under GDPR:

> Article 3: (6) 'pseudonymization' means the processing of personal data in such a manner that the personal data can no longer be attributed to a specific data subject without the use of additional information, provided that such additional information is kept separately and is subject to technical and organizational measures to ensure that the personal data are not attributed to an identified or identifiable natural person;[20]

In other words, pseudonymization guarantees that no identity can be restored if no additional information is available or accessible. However, this only applies if all sensitive attributes are taken into account and pseudonymized. Otherwise, the combination with other non-pseudonymized attributes can lead to re-identification. The principles of k-anonymity and data aggregation, which are described later in this chapter, can ensure that the sensitive data, replaced by so-called quasi-identifiers[3], remain useful for further processing and do not lead to a re-identification of the person. The k-anonymity implies that the person in the data set cannot be distinguished from at least k other individuals, even if such data is not pseudonymized. It is the sole responsibility of the data controller or data processor to protect the additional information. If leaked, it will be easy for unauthorized persons to match the data for further re-identification. Regulatory authorities often recommend using anonymized data as a priority, which does not include the storage of the additional information and reversibility of the data. However, pseudonymized data is more suitable for research purposes where the objectives of the study cannot be achieved with anonymized data, as the data must be partially or fully recovered, and each identity in the dataset must be individually distinguishable. Similarly, personal information is used whenever the pseudonymized data does not fulfill the criteria of a research study. Shin's et al. (2021) [27] paper on data pseudonymization in the medical field provides a compelling example. They highlight that if a researcher initially anonymizes data, any future additional research would require a complete data collection process. In contrast, with pseudonymous data, the researcher can easily retrieve the pseudonymized data as requested,

[3] Additional data that cannot be directly attached to the identity.

significantly saving time and resources for further research.[27]

Understanding the differences between anonymous and pseudonymous data is essential. Firstly, as mentioned above, a pseudonym can be attached to a single individual, whereas anonymity makes a person unrecognizable from anyone in the dataset.[27] Secondly, the pseudonyms can be reversible in some way, which is not the case for anonymous data. This distinction, as highlighted in the definition of pseudonymity in most sources, is underscored by the word “separation” in the GDPR definition, where it is mentioned that the “additional information is kept **separately**.” [12]

Although data separation is a crucial concept in data pseudonymization, the exact way the separation should work is not officially defined. Pseudonymity is often distinguished into irreversible, where the original data is securely replaced and cannot be retrieved, and reversible, where the pseudonyms and the original data are linked between the databases. [12] The reversible pseudonymity in connection with data separation will be covered in the next subsection. Irreversible pseudonymity, by contrast, only requires the use of pseudonymisation methods without recovery options, which are later discussed in the second part of this chapter.

2.1.3 The Concept of Reversible Pseudonymity

The reversibility of the pseudonymized data is typically managed through the data linkage process. [12] In specific scenarios, such as when the data was pseudonymized using encryption, the original data can be unveiled through data decryption, which requires using secret (private) keys. [6]

For the linkage process, the link between the pseudonym and the original value is often referred to as a pseudonymization secret, which is stored in the association table. The concealed identity can be re-identified by comparing the pseudonym in the protected data table with the association table. It’s important to note that this process must be overseen by the data controller responsible for implementing specific access restrictions, thereby ensuring responsible data handling. [5] Figure 2.1 shows an example of data linkage.

Various sources suggest different approaches for handling and storing association tables. The most common approach is described by Pommerening et al. (2014) [21] as "one-tier pseudonymization.” The first subset, previously referred to as the association table, contains identifying data, such as a pseudonym and a real name. The second subset comprises the user data, also referred to as "payload" in ISO 25237[8], which should not be pseudonymized and can be

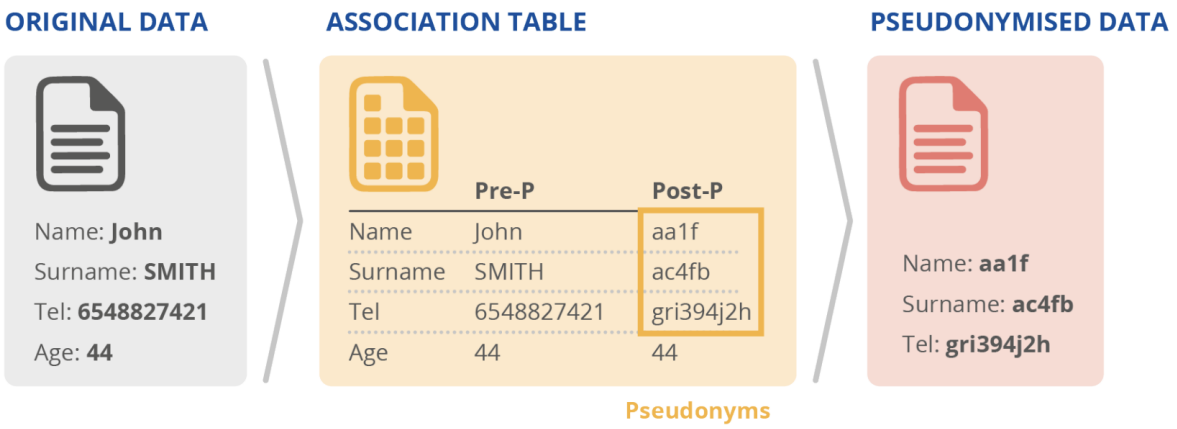


Figure 2.1: Reversible data example: data linkage [5]

used for research or other purposes.[12]

However, Pommerening et al. (2014) [21] and ISO 25237 (2008) [8] recommend deploying a newer and more secure approach to data storage: 'two-tier pseudonymization.' This method introduces a new data subset - a mapping table containing the pseudonyms' mapping from the two association tables: the identifying data and the payload data. In this respect, it is vital to point out that the pseudonyms in the identifying and payload tables must be unique, ensuring that the pseudonyms in the mapping table alone cannot be wholly re-identified without using both data subsets.[12] The 'one- and two-tier pseudonymization' concepts are illustrated in Figure 2.2. The virtual global dataset shows the original dataset combination.

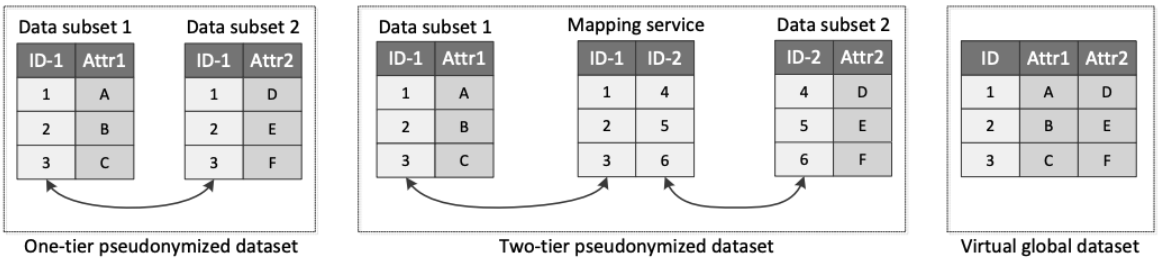


Figure 2.2: Data-tier separation: examples of pseudonymized datasets [11]

Jabeen et al. (2017) [9], on the other hand, criticize the mapping tables by mentioning the security and performance threats associated with their implementation, which is prone to disclosure attacks. The paper states that if this table is compromised, the whole system becomes

vulnerable. Additionally, the system's response time directly depends on the mapping table size and the number of association tables. Therefore, the mapping function approach, which relies on cryptographic mechanisms, is proposed as a better solution for pseudonymization.

Noumeir et al. (2007) [19] mention the advantages of the mapping function, which include lower managing and maintaining costs since less additional data has to be stored. Here, the secret key is a secure solution against unauthorized data access and can be applied to reverse data.

Finally, one of the most crucial parts of reversible pseudonymity is protecting the pseudonymization secret. The secret must be isolated from the dataset, deleted from all insecure data storage, and controlled by solid access policies. Suppose the pseudonymization secret is stored as a mapping table. If it is stored on a personal computer, it has to be additionally encrypted, and the secret key has to be protected. [6]

2.1.4 Pseudonymization of Unstructured Text

The pseudonymization of unstructured data is currently proving to be a critical and complex challenge. In contrast to linearly structured data, which can be pseudonymized with fairly straightforward solutions, unstructured data requires more pre-processing steps, such as the recognition of sensitive information, which is often context-dependent and requires advanced processing techniques such as NLP[4].

In their research, Ming and Chua (2015) [16] define a problem of polysemy and pseudonymity in unstructured text. The "Named Entities" in text processing, such as person, organization, and location names, are often not unique identifiers. Instead of mapping one entity to one identity, one must consider that there are primarily many-to-many mappings in real-world scenarios. In particular, this can be defined as polysemy when one name stands for several entities. An example from the paper [16] is that ABC can refer either to the American Broadcasting Company, the Australian Broadcasting Corporation, or even to ABC (newspaper). Another challenge is pseudonymity, where an entity can have multiple names, e.g., a full name, an acronym, or different spellings.

Eder et al. (2022) [7] describe their approach to pseudonymizing privacy-sensitive information in emails. First, they recognize privacy-sensitive information using Flair[5], a state-of-the-art NLP

[4] Natural language processing
[5] `https://flairnlp.github.io/`

and text embedding library. The SoMaJo[6] tokenizer is utilized to tokenize free text, specifically in German. Secondly, the entities found during surrogate generation are replaced by automatically generated, natural-looking substitutes. Even if part of the original data is not replaced, the few leaked identifiers in the synthetic data are difficult to distinguish. Figure 2.3 illustrates the described pseudonymization workflow of unstructured data, using email as an example.

Similar to Eder et al. [7], Yermilov et al. (2023) [29] analyze the pseudonymization of unstructured data in two steps: PII detection and replacement. Furthermore, three different systems for pseudonymization of data are investigated. The NER[7]-based system detects three predominant categories of named entities: PERSON (PER), LOCATION (LOC), and ORGANIZATION (ORG). In addition to Flair, Yermilov et al. use an embedding library called spaCy[8]. Subsequently, the found named entities are replaced by entities with similar characteristics and reassigned under some predefined constraints. The second system presented by Yermilov et al. is Seq2Seq, which uses an encoder-decoder model and defines the task of pseudonymization as sequence-to-sequence, specifically text-to-text, in one single step. LLM[9] pseudonymization is the third proposed system. It is based on a sequential chain of GPT-3, which is fast for extracting named entities, and ChatGPT, which performs a qualitative pseudonymization of the extracted named entities. The paper's results conclude that the NER-based systems with Flair are better for pseudonymization in terms of privacy preservation and downstream task performance. In contrast, LLM-based systems perform better at preserving the text's integrity. [29]

2.2 Pseudonymization Methods

This section briefly introduces the pseudonymization techniques used to implement the automatic pseudonymization library. ENISA[10] [6] classifies pseudonymization methods into single identifier pseudonymization, which includes data that can uniquely identify an individual, such as a full name or insurance number, and advanced techniques, where identifiers are dynamic and can change over time or be used in a combination to create a pseudonym.

Each pseudonymization method has its complications in terms of usage with pseudonymization, which are also considered during implementation. The possible ideas and solutions to these complications are covered in the 4 Implementation chapter.

[6] `https://github.com/tsproisl/SoMaJo`
[7] Named-entity recognition
[8] `https://spacy.io/`
[9] Large language model
[10] European Union Agency for Cybersecurity

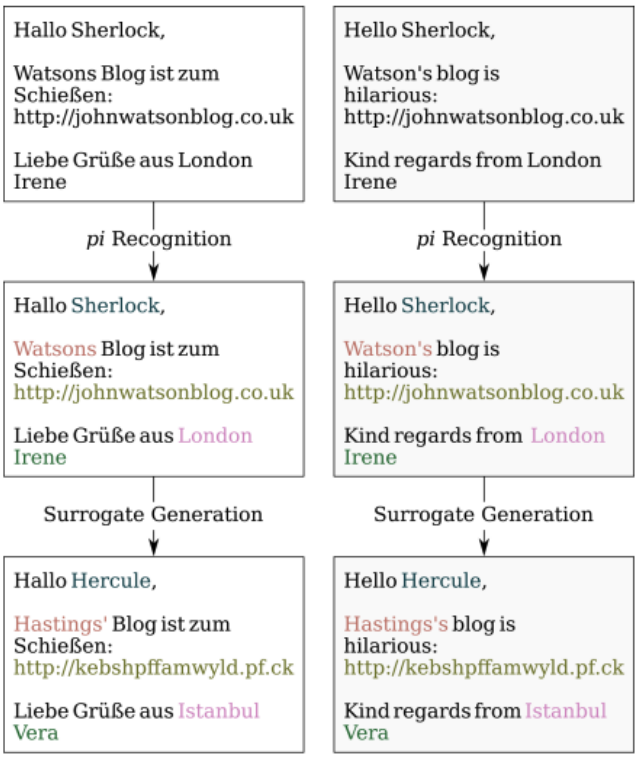


Figure 2.3: Pseudonymization workflow of emails (German original (left), English translation (right)) [7]

2.2.1 Counter

The counter method is a single identifier method that uses a monotonic counter that starts at a specific value, like 0, and is incremented by 1 with each new identity. It is the simplest pseudonymization function suitable for small and non-complex databases. Although the counter method makes it possible to de-identify data without any connection to the actual identity, scaling problems can occur if the data needs to be sorted. In this case, the new mapping tables must be recreated. [6]

2.2.2 Random number generator (RNG)

The random number generator offers a promising solution to the counter method's scalability problem. Instead of assigning the numbers incrementally, the technique defines the minimum and maximum boundaries for extracting a random number or sequence of characters. The random number can be generated by a true random number generator or a cryptographic pseudo-random number generator. However, randomness is usually associated with probability, and there is the possibility of collisions, which should be taken into account during implementa-

tion.[6]

2.2.3 Cryptographic hash function

A cryptographic hash function is one of the critical elements of modern cryptography. Due to its irreversible nature, it is often called a one-way hash function. Hash functions are frequently used for data integrity, to ensure that the data has not been altered by unauthorized means, and message authentication, to confirm the source of information. They can also be used for pseudonymization if applied correctly. The Secure Hash Algorithm (SHA) is a popular cryptographic hash function. The algorithm makes finding two different inputs with the same function output value, also called colliding inputs with $h(x) = h(y)$, computationally infeasible. This means that it is virtually impossible to find an input x where $h(x) = y$ using any known computational methods (collision resistance). [13]

More precisely, a hash function h maps bit strings of any finite length to fixed-size strings. For pseudonymization, a cryptographic hash function can be applied directly to the identifier $pseudonym = h(id)$. Although the hashed data is considered irreversible, it is vulnerable to dictionary and brute force attacks because the attackers can use precomputed tables of hash values(dictionaries) for common input data. If all data in the dataset has been hashed with the same hash function, the attacker can disclose all identities simultaneously. The ENISA report [6] provides an example of a dictionary attack on the SHA-256 hashed IP addresses, which shows that in the worst case, an attacker would need about 2 minutes to recover the IP address on an ordinary laptop with an Intel(R) Core(TM) i7-8650U CPU @ 1.90GHz processor (8 cores). [6]

This technical issue can be avoided by introducing a certain degree of randomness into the inputs of h. In this case, the hash function uses two arguments: a randomly chosen public value s, called salt, and an identifier as an input. In his book, Rosulek [25] states that the best practice is to apply a unique salt for each identity. Rather than storing $H(ID)$, choose a random salt s for each identity and store $(s, H(s, ID))$. Each identity receives a personalized hash function with salts, each of which requires a corresponding and unique dictionary for the attack. It is not necessary for the salt to be kept secret, but it is better to restrict access to it.

2.2.4 Cryptographic encryption

In cryptographic encryption, a distinction is made between symmetric and asymmetric encryption.

A cipher is a fundamental element of any cryptographic algorithm. It allows a system to transform a message into incomprehensible text (ciphertext). Encryption and decryption use public and private keys to convert an original text into cipher format and vice versa. A private key is used to encrypt and decrypt a message and, depending on the situation, is known to one or both parties involved in the message exchange. In contrast, a public key is used only for the encryption part. It is available to everyone who intends to send a secure message or verify the identity of humans and computers. [2]

Symmetric encryption, specifically block ciphers like the AES[11], is a recommended method for the robust data pseudonymization. [6] Only one secret key is required to encrypt and decrypt the data. On the other hand, asymmetric encryption requires public and private keys, which in the case of pseudonymization can be inconvenient and redundant in terms of key management and functionality.

As the block size is an essential component of block ciphers, such implementation must consider whether the size of the identifiers is smaller or larger than the size of the blocks. In this context, it must be decided whether some padding or compression should be used. [6]

2.2.5 Merkle Tree

Kermezis et al. (2021) [10] describe another pseudonymization technique based on Merkle tree, which constructs user-generated pseudonyms upon multiple user identifiers. The main advantage of this technique is that two different pseudonyms for the same identity can maintain their unlinkability. Nevertheless, data integrity can be proven without disclosing the original data. Also, Merkle trees inherit collision resistance property from the cryptographic hash function.

The Merkle tree is based on the cryptographic hash function h. The i attributes of an identity, which are first hashed to values of $h(id(i))$, form the leaf level of the tree. Furthermore, the corresponding leaf nodes are concatenated to form a hashed inner node such as $h(h(id(i))||h(id(j)))$. The root node is, therefore, composed of the inner child nodes, which, in turn, are composed of the former hashed child nodes. [10]

Regarding pseudonymization, the tree's root corresponds to the pseudonym of an identity. If a controller knows only part of the nodes and wants to prove the pseudonym, another part must

[11] Advanced Encryption Standard

be provided. Fortunately, knowing only part of the nodes does not allow computing the root result. Additionally, a unique secret key k, similar to salt, can be applied to the tree attributes if a unique pseudonym has to be created for different organizations or parties to be used. Figure 2.4 provides an example of pseudonym proving. A secret key k is applied to every attribute ID to concatenate a unique node. Suppose a controller wants to establish whether a particular user owns a pseudonym. The user must reveal the nodes marked in yellow to achieve this. [10]

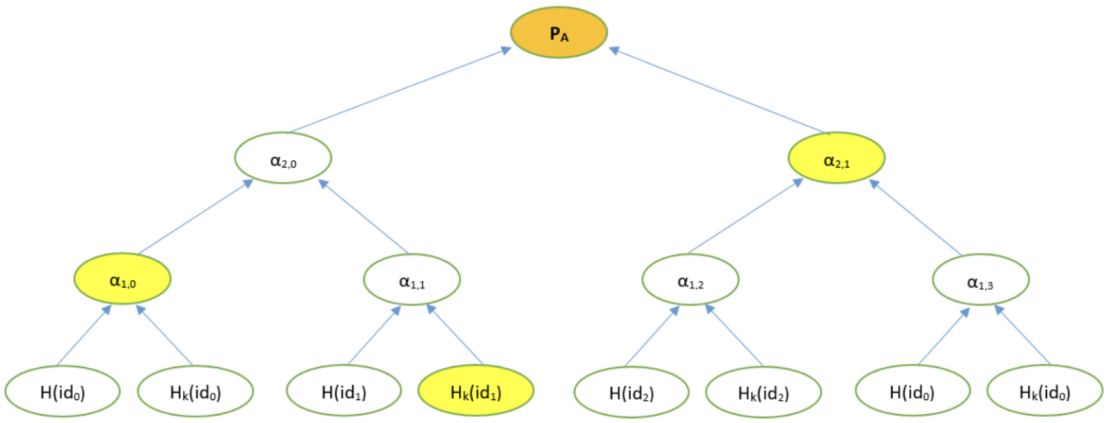


Figure 2.4: Merkle Tree: pseudonym proving
[10]

2.2.6 Aggregation

Aggregation is an anonymization technique in which the attributes of the dataset are converted into combined values. [23] It is useful in terms of data minimization when the individual values are not needed and a more general value can be used. For example, grouping the specific age values into more general age groups. Although this is not a pseudonymization method, it has been included in the library for data minimization purposes.

2.2.7 k-Anonymity

Although pseudonymizing data such as names, phone numbers, or email addresses is considered a solid data protection practice, additional data that cannot be directly attached to the identity, also called quasi-identifiers, can often be uniquely combined and easily linked to publicly available information. This allows the identity to be re-identified. [26]

Sweeney and Samarati [26] address this problem in their paper and propose k-anonymity as a solution. They define the main requirement to consider data as k-anonymous: the combina-

tion of quasi-identifiers in the dataset must be matched to at least k individuals. This can be achieved by generalization, combining similar data, and creating a more general attribute with a higher hierarchy. For instance, age can be generalized to different age groups, as described in the aggregation. Another option is to suppress data, which means removing data from the table to ensure no unique data combinations in the dataset.

It is worth noting that achieving or automating the optimal level of k-anonymity is challenging. Using a single method, such as generalization, is often insufficient and proves to be NP-hard. For this reason, mentioned methods are usually combined to achieve the best results. Meyerson and Williams [14] propose approximation algorithms for this problem. However, the running time of the algorithms is still exponential in k and at best $O(k\ log\ m)$-approximated, where m is the degree of the relation.

3 Concept

The concept of the library for automatic pseudonymization arose from the need to address the relevance of data protection for all current and future developments that use, store, or process personal data. In addition, the aim was to simplify the task of pseudonymizing data and make it more accessible for Python users. This chapter explains the concept, the other objectives of the library, the target audience, the most important features and the brainstorming process in more detail.

3.1 Goals, Purpose and Target Audience

The objective was to build a library that is effective in terms of pseudonymization and efficient in terms of usage. In this bachelor's thesis, an attempt was made to collect current practices and apply them during library implementation. Furthermore, another goal was to document the implementation and practical use of the library comprehensively.

The library must be practical for those who need to pseudonymize data but do not have the capacity or time to delve into the theoretical aspects of the process in detail. Initially, the target group consisted of users with a technical background and basic Python knowledge. However, as described later in the section 3.3 of this chapter, after implementing the graphical user interface, the focus is now on a broader audience. More precisely, the library and the topic of pseudonymization as a whole could be of interest to researchers, data scientists, and organizations that process a certain amount of personal data.

There are a number of pseudonymization tools on the market. For instance, the ERDRI.spider[1] pseudonymization tool generates pseudonyms for rare disease patients and is used on structured data. [4]. Another example is Presidio[2], an adaptable PII de-identification service for text and images from Microsoft. [15] Besides, some paid services offer pseudonymization. In

[1] https://eu-rd-platform.jrc.ec.europa.eu/spider/
[2] https://github.com/microsoft/presidio

contrast to all the previous excellent options, the automatic pseudonymization library developed for this thesis should provide the advantage of a free, generally usable solution that does not require complex configuration.

3.2 Key Features and Functionality

The library's scope enables the pseudonymization of structured data in the form of a CSV file or a Dataframe and unstructured data in the form of a txt file or a text string. For unstructured data, the focus is on the pseudonymization of names, locations, organizations, emails, and telephone numbers. Other entities can be added as extensions. In a nutshell, the library combines the following functions and capabilities:

- Choose between pseudonymization of structured and unstructured data;
- Select one of eight available pseudonymization methods;
- Create association and mapping tables by selecting the identifying data attributes for pseudonymization;
- Enabling "multi-tier pseudonymization" (see "one- and two-level pseudonymization" in the Chapter 2, section 2.1.3);
- Allow one-way or reversible pseudonymization;
- Read from the file or Dataframe;
- Write to the file or return the output;
- Encrypt the mapping tables after the pseudonymization process;
- Pseudonymization script for easier configuration;
- GUI for users with any background;
- Filter for the specific data for pseudonymization;
- Seed randomly generated outputs;
- Restore the data to its original form;
- Allow data minimization and anonymization using k-anonymity and aggregation.

3.3 Brainstorming Process

The first challenge in creating the library was to design a structure that was easy to use and did not require too many additional steps. Therefore, it was initially decided to generalize all

pseudonymization methods in one class so that the user only has to call one function with specific parameters, which performs all further processing automatically, Figure 3.1. The option has also been provided to leave most non-mandatory arguments empty and still generate the output. The output must consist of a modified source file with pseudonymized PII and the mapping files, which can be referenced later to revert the data to its original state.

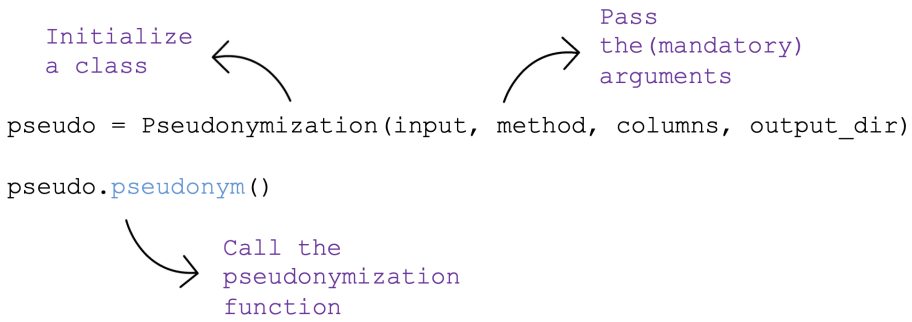


Figure 3.1: The concept of the pseudonymization class for structured data.

After testing the initial concept, it was clear that the current library setup would only be practical for developers or other users with technical backgrounds. Thus, creating a universal script that could be executed directly from the terminal using the YAML configuration file became a new goal. The idea quickly evolved into an even more universal solution - the graphical user interface, which would include all library functions, be intuitive, and cover a wider user group. However, as the primary goal was still an effective automatic pseudonymization library, extensive research had to be conducted to identify current pseudonymizing data practices. As a result, most of the practices from the Chapter 2 were used as a basis for development and combined with new ideas. The source code has also been optimized so that any external user can extend it using the template provided in the README.md file. There are clearly other pseudonymization techniques that go beyond those currently available in the library. It would therefore be beneficial to expand the library further over time.

Pseudonymity in the sense of the library is maintained by replacing the selected sensitive data with pseudonyms that are generated using pseudonymization methods. The selection of available pseudonymization methods was mainly based on the ENISA[6] recommendations. Since ENISA only provided theoretical ideas and use cases, it was necessary to dive deeper into each method and understand topics such as cryptography, hashing with salts, Merkle tree building, etc. Most methods are vulnerable to some extent to certain problems and attacks, so some preventative solutions had to be explored. Overall, ENISA concentrates on the pseudonymization of structured data, so the concept of processing unstructured data mainly stems from the combination of methods described in Chapter 2.1.4. Although unstructured data can contain

a significant amount of sensitive information, this area has not been fully explored and is currently gaining attention. Existing solutions for processing free text data listed in the literature review show promising results and describe the technologies used. However, none of the papers mentioned the specific technology used to create entities to replace sensitive information. An interesting example of the anonymization of unstructured texts was the implementation of Presidio[15], which generates fake PII data with the Faker package. The idea of a Faker method was then also introduced in the pseudonymization library to replace sensitive data in the text.

Another process that had to be thought through was the decryption of unstructured text, as NLP or Regex are unsuitable for finding the encrypted values in free-form text. Therefore, these values are also copied into a separate file, which makes it possible to compare the encrypted values with the source text and decrypt them. Next, it was essential that the user could filter the data for pseudonymization and not simply pseudonymize all the text or data at once, so the idea of specifying a filtering condition as an argument was developed. In the case of the CSV format, the user would select the column, the operation, and the value to be filtered by. For the unstructured text, the user would enter the name variations separated by commas.

3.4 GUI Concept

Since all pseudonymization functionalities were developed within the library, the graphical user interface concept primarily defines the visual interaction and navigation between the functions. First, the user must select the data type to be pseudonymized, i.e., structured or unstructured. An input form will then appear that refers to the previous selection. The user must also be able to submit the form, reset the entries, and restore the data to its original state. The form has to contain the selection of pseudonymization methods, a possibility to browse for the input files, filtering options, and other customizations. The Figure 3.2 shows a conceptual example of such a form for free text data.

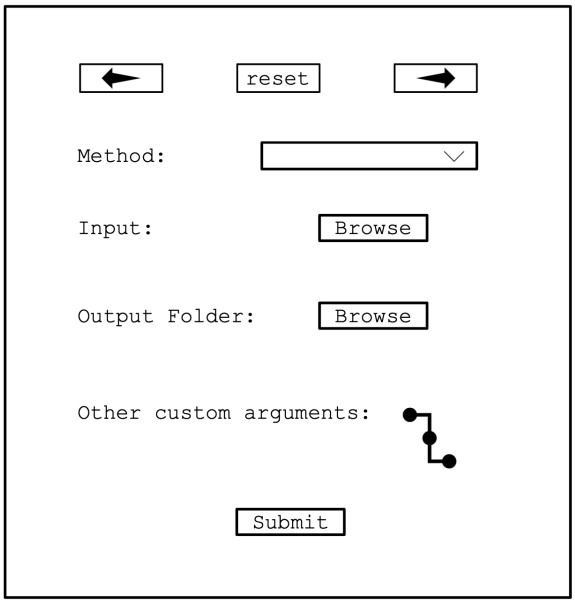


Figure 3.2: GUI Mockup for free text page.

4 Implementation

This chapter describes the library's implementation. First, the general features, the input and output, as well as the configuration requirements, are discussed. Next, the key technologies used for implementation and some motivations for these technologies are presented. The program flow describes the program's step-by-step execution and explains the pseudonymization methods in detail. Finally, some use cases for the technology are suggested.

4.1 About the Library

The library implementation is written in Python, a high-level, general-purpose programming language that is practical in terms of supporting paradigms such as object-oriented, structured, and functional programming. The language is widely used in the data science community and is well suited for the pseudonymization library's target group. Python's popularity has resulted in many open-source libraries that make development more efficient and remove complexity from the source code. The implementation was done using Python 3.12.1 and PyCharm IDE. The library has been uploaded to GitHub so that anyone can clone[1] it to the local repository.

A prerequisite for using the library is that Python is installed. In addition, a virtual environment is recommended to install all required packages in an isolated location. For example, venv[2] is available from Python version 3.3 as a standard module for creating virtual environments. The required packages are frozen in the requirements.txt and must be installed with pip. To launch the script or GUI, the user can run it from the terminal, but prior setup of the virtual environment and the required packages is still necessary. To be able to use the individual classes and functions in their code, the user must first import them into the relevant file.

The only mandatory parameter for a successful result with unstructured data is the actual data to be pseudonymized. Depending on the data type, this can take the form of a path to the

[1] `https://github.com/oleksapopovych/pseudPy`
[2] `https://docs.python.org/3/library/venv.html`

input file, a data frame or a text string. The names of the target columns are also mandatory for structured data. In addition, the user can pass the pseudonymization method, which is 'counter' by default; the output directory path, if output to files is required; specify whether mapping files are necessary and whether the mapping must be encrypted for more secure storage. For unstructured data, the user can also specify other arguments, such as selecting the PII type or the PII word patterns to be pseudonymized. However, all additional arguments are optional. All possible arguments are listed in the library documentation in the Appendix 1.

The output of the library is provided in two ways: It is returned as a list with the pseudonymized data together with the corresponding mappings as a data frame or, if the output path is given, as files in the specified directory. The returned values can be used for further data manipulations. The Table 4.1 contains some possible name patterns for the output files. The library is defined by the classes Pseudonymization, Mapping, Aggregation, KAnonymity, and Helpers. The Helpers class is the only class that cannot be used independently, as it only defines utility functions for data processing.

If the pseudonymization is performed via $script_\{x\}.py$, the configuration YAML file receives the same input parameters, but they must all be specified explicitly. In this case, the output can only be delivered as files. When executing $script_\{x\}.py$ via terminal, the absolute path to the YAML file must be passed as an argument. Alternatively, the YAML file must be in the same directory as $script_\{x\}.py$, in which case only the name of the configuration file is specified. The user can also leave unnecessary parameter entries empty by assigning the value $null$ to the arguments. A total of three scripts are available in the implementation: $script_pseudonym.py$, for the pseudonymization of the data; $script_anonym.py$, for the k-anonymization and aggregation; and $script_check_k_anonymity.py$, for the validation of the k-anonymity of the data. The corresponding example configuration files $config_\{x\}.yaml$ are also located in the $pseudPy$ folder of the library.

As already defined in the literature review, two-tier pseudonymization is recommended over one-tier pseudonymization in order to store additional information for later data reversal. In this solution, it was decided to allow multi-tier pseudonymization so that the data controller can determine which data should be pseudonymized and to what extent. In the case of structured data, the user can select several columns and receive several mapping tables. The same applies to unstructured data, where the user can choose several data types for pseudonymization. If the method for de-identification is encryption, no mapping tables are output, and the secret keys are provided as pseudonymization secrets.

File Name	Description
output.csv	Pseudonymized/Aggregated structured data.
mapping_output_{x}.csv	The mapping between the original value and a pseudonym. x is a column or entity name.
text.txt	Pseudonymized free text data.
secure_key_{x}.txt	A secret key that is used for encryption/decryption. x is a column or entity name.
decrypted_output_{x}.txt	Decrypted data. x is a column name.
reverted_output.csv	Reverted to original csv data.
reverted_text.txt	Reverted to original free text data.
k_anonym_output.csv	k-anonymized structured data.

Table 4.1: Possible Output Files

4.2 Main Technologies

Pandas vs. Polars

The first version of a library was implemented with the widely known Pandas[3] library. The library is well-documented, enables flexible structured data manipulation, and is based on Python. However, after performing unit tests with different configurations of varying complexity and data volumes, it became clear that Pandas is inefficient when it comes to larger data sets and additional complex configurations, such as the encryption of mappings in addition to pseudonymization and subsequent decryption. Therefore, it was decided to migrate the implementation to Polars[4], which offers a comparatively high-speed analysis of the data sets. Polars, which is written in Rust, has inherited its ability to compile to machine code without an interpreter. It is also based on the multithreading engine, which enables parallel execution of operations. In addition, lazy evaluation allows queries to be executed only when needed. [22] On the other hand, Pandas offers a number of functionalities that are not available to Polars users. For example, Pandas' data aggregation function cut in the $group_num()$ function was a better choice in terms of effectiveness and performance than writing it from scratch with Polars. In addition, Polars offers the possibility of converting a data frame from Polars to Pandas and vice versa with to_pandas and $from_pandas$ operations so that both libraries can be used in a single solution. The tests mentioned in section 5.4 were implemented to compare the performance of the two packages and choose the best option for the current development.

[3] `https://pandas.pydata.org/`
[4] `https://pola.rs/`

Regex

Regex, or regular expression, is a characters sequence that defines the string pattern to be matched in a text. In Python, the re[5] package can be used to work with regex patterns. In the library implementation, the predefined regex matches the email and phone number patterns in texts for further de-identification.

Faker

The Faker[6] package is used in the library to create highly realistic fake data as pseudonyms. It is able to generate fake names, locations, organization names, phone numbers, emails, etc., which is especially relevant for plain text data where it is essential to keep the text contextually consistent and readable while hiding the true identities.

spaCy

The Natural Language Processing library spaCy[7] recognizes PII in unstructured text within the implementation. It is a free, open-source solution written in Python and Cython designed for production use. spaCy supports tokenization, part-of-speech (POS) tagging, entity linking, named entity recognition (NER) and much more and is effective in terms of the capability of its predefined models and the processing of large amounts of data at high speed.[1] Also, its simplicity and accuracy made it a solid choice for the pseudonymization library, where the model named $en_core_web_sm$ is used to recognize PII such as names, places and organizations in English-language texts. With rule-based matching, the user can specify user-defined word or phrase patterns that the SpaCy Matcher should search for in the text. In the case of pseudonymization, the user can filter out only the information that must be pseudonymized and retain the other entities unchanged. For example, this allows the user to specify that all occurrences of the name Max Mustermann or of the Project X must be found and then de-identified. Below is an example of rule-based matching, where LOWER is the lowercase form of the token text:

Listing 4.1: Example of Rule-based matching

```
patterns = [[{"LOWER": "Max"}, {"LOWER": "Mustermann"}],
            [{"LOWER": "Project"}, {"LOWER": "X"}]]
```

YAML Config File

The YAML format offers good readability and easy maintenance of the configuration file. In

[5] https://docs.python.org/3/library/re.html
[6] https://faker.readthedocs.io/en/master/
[7] https://spacy.io/

addition, the YAML parser is integrated into Python, providing extensive support for its reading and writing. The user of the pseudonymization library can execute the script.py directly via the terminal by specifying config.yaml as an argument to automatically de-identify the data. Within the YAML, the user must specify all the arguments previously defined in the documentation in the Appendix 1. However, if the arguments are not required, the user can set the value $null$ next to them.

tkinter

The Tkinter[8] package, a standard Python interface building package, enabled the pseudonymization library to integrate into an actual GUI. To be more precise, Tkinter consists of several modules, and the main module used for the current implementation is called Tk. This module contains commands for creating and managing GUI widgets and events. The widgets may be input fields or buttons, while events react to user input and possible changes in the program. Other key features of the package include geometry management, such as the placement of widgets in the grid, widget configuration options that allow the appearance and behavior of the widgets to be changed, etc.

4.3 Program Flow

The program flow describes the sequence in which the code is executed, the classes and functions used for the purpose of pseudonymization and anonymization of data. The pseudonymization methods and the implementation of reversibility along with other important features are also described in more detail.

4.3.1 Pseudonymization

Flowchart 4.4 shows the program flow of the $Pseudonymization$ class. First, the user must pass the required arguments mentioned in the documentation in the Appendix 1 to the $Pseudo-nymization$ class. The $Pseudonymization$ class then initializes the arguments. The user must now call the corresponding function, depending on whether the data is structured or unstructured. For example, a $pseudonym()$ function is called if the data is structured. The function removes columns and rows that are completely empty and checks whether the number of rows is at least one. If the selected pseudonymization method, map_method, exists, the $handle_map_tiers()$ function is also called, which takes over the data manipulations and the final appearance of the output files or the returned values. However, the actual action happens

[8] https://docs.python.org/3/library/tkinter.html

in the $Mapping$ class, which is called at the very beginning of the $handle_map_tiers()$ function. To avoid repetition of conditional statements in form of if-then-else, it was decided to collect all pseudonymization methods in a single dictionary, where the keys are the names of the methods and the values are the corresponding functions that generate the pseudonyms. A particular function is called within the dictionary depending on the method selected.

Suppose the preferred method is "hash-salt." Since all pseudonymization methods are collected as functions in the $Mapping$ class, the $hash_salt_tier()$ function is now called from this class. The "hash-salt" method creates pseudonyms by hashing the initial values with the SHA-256 algorithm and using a randomly generated value in hexadecimal encoding as the salt. The function returns the Series of pseudonyms that are later inserted and further processed in the $handle_map_tiers()$. The column passed as a $map_columns$ argument is then replaced by the Series of pseudonyms in the initial file; other columns remain unchanged if it was not explicitly requested to pseudonymize those. Simultaneously, the same sequence of pseudonyms is inserted into the mapping file along with the original column. If $encrypt_map$ is set to $True$, the original column in the mapping file is also encrypted, and the secret key is generated accordingly. With the $patterns$ parameter, the user can specify the condition for data filtering by setting values such as the column to filter, the condition operator, and the value to filter by. In contrast to this process, the "decrypt" method is executed slightly differently. It is important that the secret key generated during encryption is stored in the same folder as the specified path $output$ or in the same folder as the associated Python script. In addition, only the parameters $input_file$, $output$ and the column name $map_columns$ are required for decryption. The user can also use the $patterns$ argument to filter out only the specific pseudonym that is to be uncovered.

Now, consider another scenario in which the input data is unstructured in the form of free text. In this case, natural language processing is first initialized by loading the English language model using spaCy at the beginning of the $nlp_pseudonym()$ function in the $Pseudonymization$ class. Then, the PII recognition takes place: The $entity_mapping()$ finds all emails and telephone numbers with the help of Regex and all named entities via spaCy entity identification and collects them in the dictionary. The entities found are transferred to the $Mapping$ class, responsible for creating the pseudonyms. The entities to be pseudonymized can be specified in a list pos_type, e.g., "Names," "Locations," "Organizations," "Emails," and "Phone-Numbers." Alternatively, the user can set the parameter all_ne to $True$ and only pseudonymize names, places and organizations. The original entities are later replaced in the text by the generated pseudonyms. If the $patterns$ parameter is provided, the specified words and phrases are also pseudonymized. However, these are classified as "Others" in the pos_type list. If the selected pseudonymization method is "faker," the entities with type "Others" are replaced by randomly

generated words. For other entity types, there are predefined faker instances, such as false names, locations, emails, etc. Otherwise, if a different method is used, the pseudonyms for all entities are generated in the same way. A file with the pseudonymized text and the CSV files with the mappings are finally created. Similar to the structured data format, the original data can also be encrypted in the mapping file, and the files are only output if the output directory has been passed as an argument; otherwise, the data is just returned within the system.

4.3.2 Aggregation and k-Anonymization

In terms of data minimization, the user can aggregate and k-anonymize data. Both functionalities were developed separately from the $Pseudonymization$ class and were designed to be irreversible. Flowchart 4.5 shows the program flow of the $Aggregation$ class. Also, the individual data entries must be indistinguishable from other identities, so the concept of pseudonymity does not hold here. Aggregation is used to group entries into more general values by taking arguments such as a data frame or a path to the input file, the column to be aggregated, and the aggregation method passed as a list together with the range of the generalization. The user can choose between the aggregation of the numerical values and the DateTime aggregation. For example, let's look at the aggregation of numerical values: The $Aggregation$ class is initialized, and the $group()$ function is called, leading to the $group_num()$ function responsible for the actual aggregation. Depending on the selected range, the bins, i.e., the criteria to bin by and the labels for the returned bins, are created here, and the data is split into value ranges. The output is an aggregated CSV file or a returned data frame. Date values are aggregated similarly, but the values are first generalized to years, and only then are the years divided into groups.

In contrast, the $k_anonymity()$ function first generalizes the values of a selected column by reducing the granularity of the data based on the depth specified for each column. The value is essentially rounded down (floored) to the nearest multiple of 10^{depth} by the $generalize()$ function. Flowchart 4.6 shows the program flow of the $KAnonymity$ class. Here, the $KAnonymity$ class must first be initialized with the input parameters. Although no other non-numeric columns are changed by default, the string values can be completely removed if the parameter $mask_others$ is set to $True$. Finally, the values are grouped and filtered by data with at least k data records. The output is a k-anonymized CSV file or a returned data frame. Aggregation and k-anonymization can be combined to achieve optimal results in data minimization. The functions $k_anonymity()$ and $is_k_anonymed()$ are adapted from the approach presented by Near and Abuah [17] in their book on differential privacy programming.

4.3.3 Reversibility

A $revert_pseudonym()$ function in the $Pseudonymization$ class can revert the structured data to its original state, while the $revert_nlp_pseudonym()$ function reverts the pseudonyms in the free text data. The pseudonymized data frame df and the mapping data frame $revert_df$ as well as $map_columns$, the name of the column with the original data in the mapping table, are required to complete the reversal. All pseudonyms from the pseudonymized file are compared with the mappings and replaced with the initial value. If only a part of the pseudonyms needs to be uncovered, the user can filter only a part of them in the mapping file and pass the filtered mapping on for reversal. As already mentioned, encryption is the only method that can be reversed with the same functions as it was pseudonymized, but with map_method equal to 'decrypt.'

4.3.4 Pseudonymization Methods

All of the following pseudonymization methods are also summarized in Table 4.2 for a better overview. The pseudonymization method must be predefined by the user when the $Pseudony-mization$ class is initialized by passing the argument map_method with the name of the method as a string.

counter

A monotonic counter that starts at a specific value, e.g., 0, and is incremented by 1 for each new identity. The value $counter_start$ defines the start of the enumeration and is updated for each column depending on the last inserted number for the previous column.

random1, random4

A random1 method generates a UUID[9], $uuid1()$, from a host ID, a sequence number, and the current time, while the random4 method generates an entirely random UUID, $uuid4()$. Both methods use the uuid[10] package. In addition, a $seed$ argument can be passed to make the results predictable and repeatable.

hash

The hashing method generates a series of pseudonyms by applying the SHA-256 algorithm to the hexadecimal encoding of each entry value. The hashlib[11] package provides the hashing

[9] Universally unique identifier
[10] `https://docs.python.org/3/library/uuid.html`
[11] `https://docs.python.org/3/library/hashlib.html`

functionality.

hash-salt

With the hash-salt method, pseudonyms are created by hashing the initial values with the SHA-256 algorithm and using a randomly generated UUID value, $uuid4()$, in hexadecimal coding as a salt. The hashlib package provides the hashing functionality, while the uuid package provides the random value.

merkle-tree

The method combines all identifiers of each structured entry row in a Merkle tree. The pseudonyms are defined as the unique root values of the individual corresponding Merkle trees. In addition, hashlib is used for hashing with SHA-256. The Merkle tree algorithm was adapted based on the existing Mix Merkle tree[12] project on GitHub.

encrypt/decrypt

A secret 256-bit key is generated for encryption, which can be retrieved later for decryption. The symmetric encryption algorithm AES[13] is based on the block cipher, which divides the message into blocks with a size of 128 bits. The ECB[14] encryption mode ensures that the same plaintext value results in the same ciphertext value, while encryption is performed block by block with a single key. The empty blocks are filled with PKCS7 padding, which allows block sizes between 0 and 255 bytes. The result is encoded in UTF-8 to make the encoding appear better in the output data. During decryption, the cipher decryptor is executed, and the unpadded function is used to restore the data. All these functions are imported from the cryptography[15] package. The solution is a combination of the practices described in the documentation of the cryptography package and the implementation of AES with padding published by Vignya Durvasula[16].

faker

The faker method can only be used on unstructured data. Depending on the entity type, the method automatically replaces the input value with the fake name of the corresponding entity. For structured data, the specific faker type that matches the desired fake name type must be selected manually. For example, if a column in the data frame consists of place names, the user can choose the "faker-loc" method to create fake location names. Other methods are "faker-

[12] `https://github.com/onuratakan/mix_merkletree`
[13] Advanced Encryption Standard
[14] Electronic Code Book
[15] `https://cryptography.io/en/latest/package`
[16] `https://www.askpython.com/python/examples/implementing-aes-with-padding`

name" for the full names, "faker-email" for the emails, "faker-phone" for the phone numbers and "faker-org" for the organization names. The fake instances are produced with the Faker package. Similar to the Presidio implementation, where the PII data is replaced with fake data using Faker, this idea was also implemented in the pseudonymization library.

Method	**Description**
'counter'	Monotonic counter, e.g., 0, 1, 2, 3 ...
'random1'	UUID from a host ID, a sequence number, and the current time.
'random4'	Entirely random UUID.
'hash'	Apply SHA-256 algorithm to the hexadecimal encoding of each entry value.
'hash-salt'	Apply SHA-256 algorithm and the randomly generated UUID value in hexadecimal coding as a salt.
'merkle-tree'	Use the unique root values of the individual corresponding Merkle trees as pseudonyms.
'encrypt'	Encrypt data with the symmetric encryption algorithm AES.
'decrypt'	Decrypt data with the symmetric encryption algorithm AES. Make sure that the keys are in the output folder or in the same folder as the script.
'faker'	Replace data with fake data in unstructured texts. Only available for the pos_types "Names", "Locations", "Organizations", "Emails" and "Phone-Numbers".
'faker-{x}'	Replace data with fake data in structured data sets. Available methods: "faker-name", "faker-loc", "faker-org", "faker-email" and "faker-phone".

Table 4.2: Pseudonymization Methods.

4.3.5 GUI

The graphical user interface can be seen as an exemplary use case for a library and, simultaneously, as a simplified way to achieve automatic pseudonymization for a broader user group. At the beginning, the homepage welcomes the user and suggests selecting a form of data to be pseudonymized: structured or unstructured. Depending on the answer, a new window opens containing widget entries corresponding to the chosen data form. Alternatively, the user can access the k-anonymization of the structured data directly via the homepage. Next, consider selecting the 'structured data' option. In the Figure 4.1 on the left, the user can now select

the pseudonymization method, browse the input and output paths, filter the data, choose the columns to be pseudonymized, enable or disable the options for including and encrypting the mapping, and specify the seed for randomly generated pseudonyms. Taking a closer look at the Filter data option, the user must select the column to filter by, the operation and value to compare the column values to, and the type of value, e.g., integer or string. The type was added because the tkinter entries always return strings, which is impractical for numerical comparisons. Once the form has been completed, the user can submit it by pressing the "Go!" button. The de-identification results can then be found in the specified output directory. In the header of the window, there is the "Go Back" button, which allows the user to return to the homepage; the "Reset Inputs" button, which resets the input values to the initial state; and the "Revert Data" button, which leads to a new window in which the data can be reverted to the original state.

In the right-hand window 4.1 for unstructured data, the user can also select the type of personal data to be pseudonymized by either selecting the "All Named entities only?" checkbox or selecting a specific kind, e.g., "Names," "Locations," etc. If the option "All Named entities only?" is set to $True$, names, locations and organizations are automatically selected in the "Type of Data" list. The "Filter for" offers the option of selecting only certain word combinations separated by commas that are to be pseudonymized. In this case, the user should additionally select the value $Others$ in the “Type of Data” entry. To restore the data (see Figure 4.2, left), the path of the mapping file must be specified in addition to the input and output paths. The column name is the column’s name in the mapping file that contains the data to be restored. If not everything is to be disclosed, specific pseudonyms can be provided, separated by commas. In the case of k-anonymity (see Figure 4.2, right), k must be specified. In addition, data aggregation is available to minimize the data, whereby the user must select the columns to be aggregated and the range of aggregation groups, the Gap.

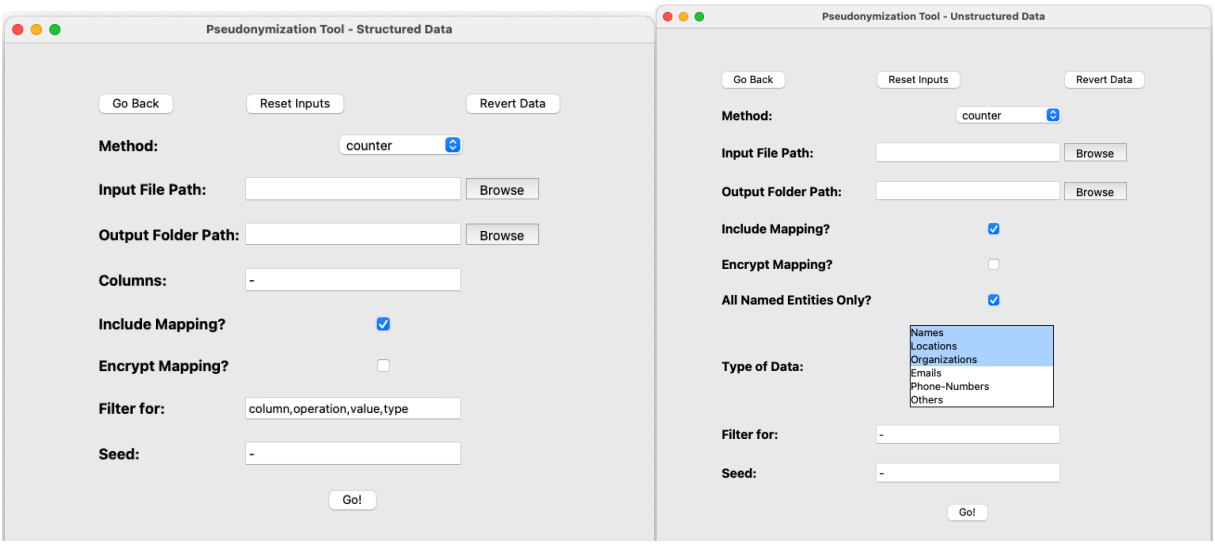


Figure 4.1: Left: Page for Structured Data; Right: Page for Unstructured Data

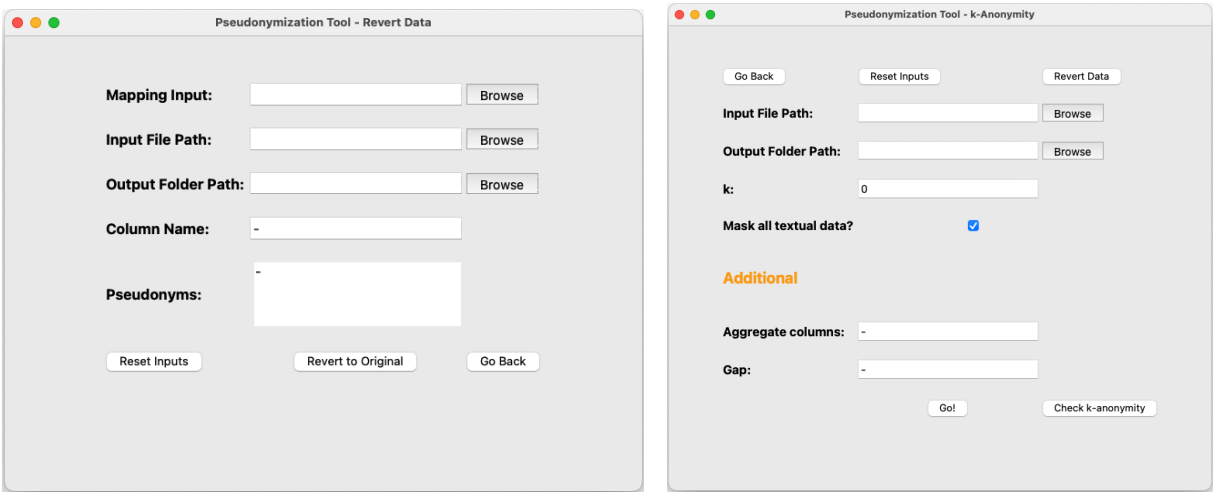


Figure 4.2: Left: revert the data; Right: k-anonymize the data

4.4 Use Cases and Scenarios

This section discusses various application scenarios for the pseudonymization library. First, an overview of these scenarios is given, and then each scenario is explored in detail with example inputs and outputs.

4.4.1 Overview of Scenarios

1. Internal Data Storage

- Structured data.
- Personal data collected by the institution for internal use.

2. External Data Exchange and Disclosure of Information
   - Structured data.
   - Sharing data with external partners while protecting identities.
   - Disclosure of information on request.
3. Storage of Confidential Meeting Reports
   - Unstructured data.
   - Secure storage of report data and ensure the readability of reports after pseudonymization.
4. Data Analysis and Research
   - Structured data.
   - Request the data for the statistical analysis. There is no requirement for individual figures but for the overall picture.

4.4.2 Scenario 1: Internal Data Storage

The first and most common possible scenario is that the personal data is collected by the institution and is intended for internal use only. In this case, the data can be pseudonymized for more secure storage to avoid a data leak in the event of an external attack or security incident. Since the actual data and the mappings, known as pseudonymization secrets, are stored separately, the likelihood of all personal identifiers being exposed is lower than usual. More secure pseudonymization methods, such as hashing with salts or Merkle tree pseudonyms and other methods, make the identifiers irreversible and thus guarantee more secure storage.

A more concrete example could be a small software company that collects personal information for its statistics, forecasting, and ad planning. According to GDPR, the company must ensure the protection of its customers' data, and pseudonymization should be considered for this purpose. This is where the automated pseudonymization library comes into play. The data controller extracts the data from the database and converts it into the form of Polar's data frame[17]. The data controller decides that the most convenient method in their case is to use a unique, randomly generated ID as an identifier-pseudonym for their users. The "random4" pseudonymization method provided by the library meets the defined expectations and is chosen to create pseudonyms. The controller does not require the output to files; therefore, the

[17] `https://docs.pola.rs/user-guide/io/database/`

output is returned within the system and uploaded to the database later. Also, the controller decides to pseudonymize two personal identifiers so that two columns are selected for processing. The input data consists of five identifiers: name, country, gender, salary, and job title. This is how the controller could set up the library functions, Listing 4.2:

Listing 4.2: Example Code - Use Case 1

```
import pseudPy.Pseudonymization as pseudPy
pseudo = pseudPy.Pseudonymization(
        df = df,                    # imported Polars dataframe
        map_method = 'random4',
        map_columns = ['name', 'job_title']
        )
pseudo.pseudonym()

# Output: [pseudonymized_df, mapping_name_df, mapping_job_title_df]
```

Listing 4.3: Example Output - Use Case 1

```
# pseudonymized_df:
Index_name,country,gender,salary,Index_job_title
3aab6bOb-7e33-47ce-908-9c2602,China,Female,162080,eb4746ff-139b-4d1f-8676-eda403

# mapping_name_df:
Index_name,name
3aab6bOb-7e33-47ce-908-9c2602,Emily White

# mapping_job_title_df:
Index_job_title,job_title
eb4746ff-139b-4d1f-8676-eda403,Product Engineer
```

The output includes the pseudonymized data frame and two corresponding data mappings, Listing 4.3. The controller decides to store the data records in separate databases.

4.4.3 Scenario 2: External Data Exchange and Disclosure of Information

If personal data is requested for external research purposes or other applications, the first data frame in the output of the previous scenario, $pseudonymized_df$, i.e. the one in which the names are replaced by pseudonyms, can be shared externally. If the researcher requires further information about a particular user's job title, the controller may disclose this. Example code for the reversal of data, Listing 4.4, and the output example, Listing 4.5:

Listing 4.4: Example Code - Use Case 2

```
import pseudPy.Pseudonymization as pseudPy
df = pl.read_csv(input_file)     # import the pseudonymized data
```

```
revert_df = pl.read_csv(mapping_file)   # import the mappings
pseudonyms = ['eb4746ff-139b-4d1f-8676-eda403']     # specify the pseudonyms if only
    these are to be revealed

pseudo = pseudPy.Pseudonymization(
        map_columns = 'job_title',
        df=df
        )
pseudo.revert_pseudonym(revert_df, pseudonyms)
# Output: DataFrame with revealed pseudonyms
```

Listing 4.5: Example Output - Use Case 2

```
Index_name,country,gender,salary,job_title
3aab6bOb-7e33-47ce-908-9c2602,China,Female,162080,Product Engineer
```

4.4.4 Scenario 3: Storage of Confidential Meeting Reports

In the following scenario, the company holds a conference with its major customers and creates notes for the meeting. The notes contain the customer's personal data and confidential information about the company. The company wants to guarantee the secure storage of such information and ensure the report's readability. Therefore, it is concluded that the "faker" method is suitable for pseudonymizing the reporting data, and the following configurations are suggested for the YAML file:

Listing 4.6: Example Configuration YAML - Use Case 3

```
map_method: faker
input_file: path/report.txt
output: path/output
mapping: true
all_ne: true
encrypt_map: false
map_columns: null
patterns: null
pos_type: null
seed: null
```

Once the configuration file has been defined, the data controller can execute the pseudonymization script with `python srcipt.py config.yaml`. The configuration suggests de-identifying all personal attributes that are categorized as named entities, including names, locations and company names. For example, the sentence from the report:

`'The finance department of ABC, overseen by Emily White in Mumbai, reports a 25%`

```
increase in operational efficiency.'
```

is converted to

```
'The finance department of Mccoy and Sons, overseen by Albert Mcpherson in South
Crystal, reports a 25% increase in operational efficiency.'
```

4.4.5 Scenario 4: Data Analysis and Research

Next, the company's research department requests the data for statistical analysis. They do not require individual figures but the overall picture. Before sharing the user data, the controller, therefore, decides to k-anonymize the data so that no identity can be revealed. The controller configures the anonymization within the GUI, as shown in the Figure 4.3.

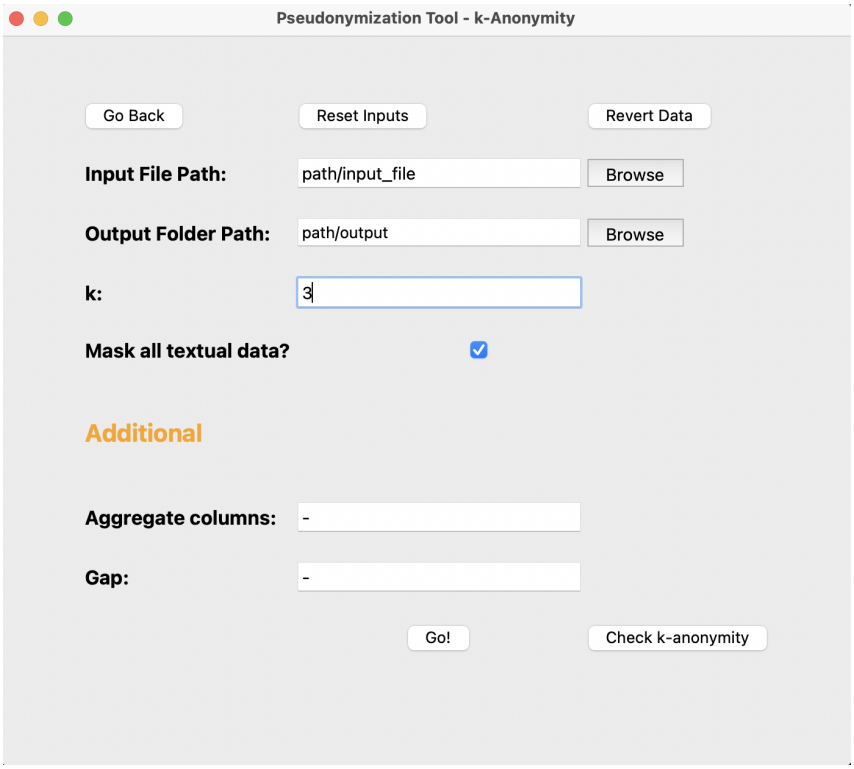


Figure 4.3: Configuration Example - Use Case 4

The output file consists of data that is identical for at least three individuals within the dataset. Hence, k equals three. Other data values are removed from the data set. In this scenario, the aggregation parameters are skipped. The controller validates the 3-anonymity by selecting the output data as the input file and pressing the "Check k-anonymity" button; as expected, the message "Success" is displayed. If the user attempts to validate 3-anonymous data with

a k value of 4, the message "Success" may still be displayed, as a k-anonymized data set must have at least k identical values or above. In this particular case, the message "Failed" is displayed when the 4 anonymity is checked. To achieve more precise results, the user can use the library argument $depths$ and test different depths for the data distribution. An example input and output can be seen in Listing 4.7, where the salary data occurs at least three times, i.e. are not unique identifiers. In the example dataset, however, the salary values were carefully chosen so that none of them were lost after the values were rounded and the 3-anonymity was retained. In the real world, if some of the values remain unique even after generalization, they are usually removed.

Listing 4.7: Example Input/Output - Use Case 2

```
"Input":
name,salary
Maren Colhoun,131439
Donica Bullas,197343
Ingamar Gallandre,66210
Alon Boots,131460
Micaela Heintz,197320
Timmy McCullough,66239
Clare Halbard,131495
Omar Idel,66200
Skylar Mabbitt,197340
```

```
"Output":
salary
131400
197300
66200
131400
197300
66200
131400
66200
197300
```

4.5 Documentation

To ensure the quality of the documentation, the library was documented with the Sphinx[18] package. This package uses reStructuredText as a markup language and is handy for converting inline Python docstrings into HTML web pages and PDFs. The documentation can be found in the Appendix 1 as a PDF or in the GitHub repository in HTML format[19].

[18] https://www.sphinx-doc.org/en/master/index.html
[19] https://github.com/oleksapopovych/pseudPy/blob/main/pseudPy/docs/build/html/index.html

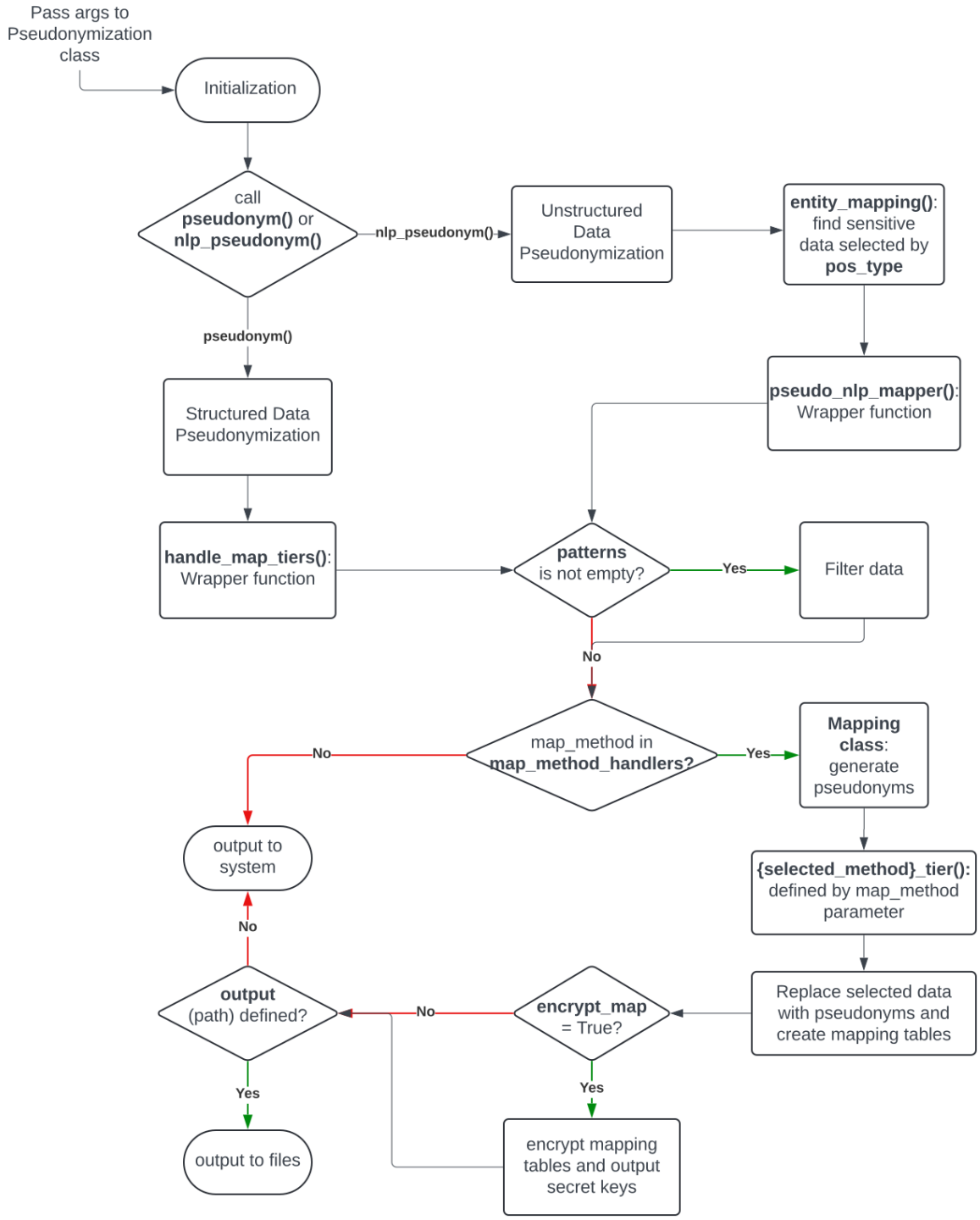


Figure 4.4: Flowchart: Program Flow of Data Pseudonymization.

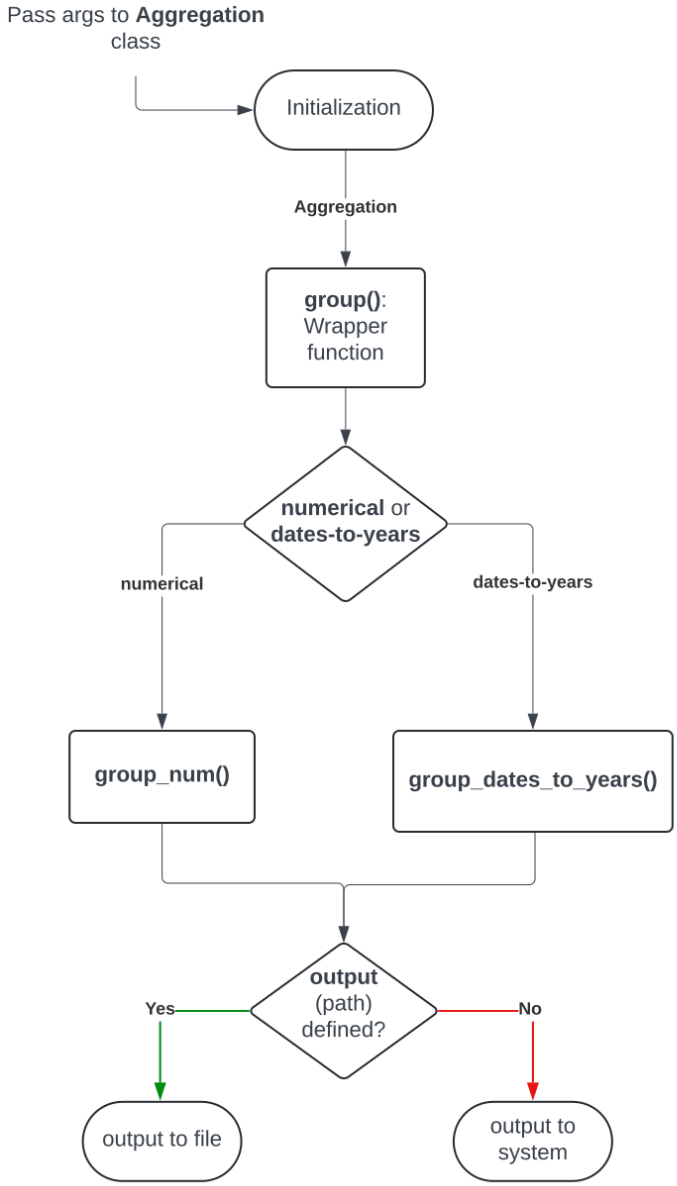


Figure 4.5: Flowchart: Program Flow of Data Aggregation.

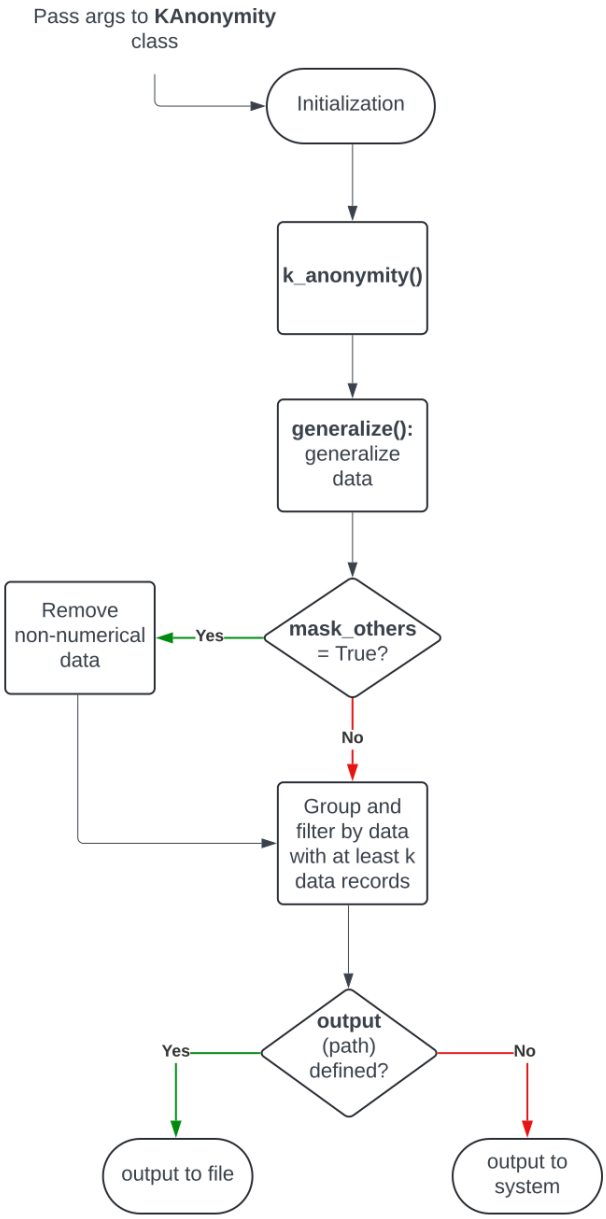


Figure 4.6: Flowchart: Program Flow of Data k-Anonymization.

5 Testing

Several tests were conducted to evaluate the pseudonymization library's functionality, efficiency, and effectiveness. The present chapter discusses these. The first part includes a functional test of the library and the graphical user interface. The second part contains a runtime analysis of the individual library functions. Finally, the third part presents and analyses the results for k-anonymization.

5.1 Functionality Testing

The pseudonymization library was mainly tested with unit tests developed for structured and unstructured data forms. The tests cover all the library's possible functions, including different pseudonymization methods, input forms, encryption/decryption, data restoration to the original, aggregation, k-anonymization, and invalid data reading. The developed unit tests must guarantee that the library works after any new changes are applied and show examples of library use cases. The tests compare the expected results stored in the $test_files$ folder in the library with the actual results obtained when a particular function is executed. The unit test is considered successful if the expected and actual results match. The user must ensure that if a new function is added to the library, all available unit tests must still be completed successfully, and the new unit test must be developed for the new functionality.

Regarding the input test data, the unstructured mock input data was created with ChatGPT 4o (Model status date: 30.05.2024). The data represents a fictitious work report with personal information such as full names, cities, project names, phone numbers, etc. To test a more realistic and extensive example, a Wikipidea page[1] about a famous businessman was also included in the test data. An online data generator called Mockaroo[2] was used to create structured mock data similar to free text data with personal information but in tabular form. In addition, the expected output data was defined to serve as an example of the correct output. In contrast to

[1] `https://en.wikipedia.org/wiki/Elon_Musk`
[2] `https://www.mockaroo.com/`

other functionalities, k-anonymity was tested using an additional function, $is_k_anonymized()$, which checks whether the dataset satisfies k-anonymity. Furthermore, tests with invalid data, such as reading an empty dataset or encrypting a value of type None, were helpful in understanding the weak points of the library.

Most of the unit tests contain hard-coded parameters. The introduction of the config YAML file made an exception, as it can be configured dynamically without changing the code itself. It should also be emphasized that most pseudonymization methods have a random nature; therefore, the actual result is usually not repeatable and problematic in terms of meeting the expected results. To solve this problem within the unit tests, the seed option has been added to ensure a more consistent result.

The functionalities of the graphical user interface are based on the functions of the library, which are tested with unit tests. Therefore, the GUI tests included more of the front-end and back-end functions of the GUI widgets rather than the pseudonymization library. The tests were exploratory, i.e., the test use cases were created simultaneously as the tool was used, and the bugs found were fixed in parallel. Interestingly, the GUI tests also helped to uncover some issues within the library that were not obvious during the unit tests. Thus, the development of the GUI was also a valuable example of using the library in real-life scenarios and understanding its limitations. One limitation was that the data filtering option was only implemented for one specific condition, so the user had to iteratively initialize the pseudonymization to apply multiple filters for structured data. In addition, the k-anonymization of data is only intended for numerical data so that the rest of the data remains unique and the user can be re-identified with the remaining attributes. It is, therefore, recommended to use k-anonymization or aggregation as a preparatory stage for pseudonymization if the user does not want to remove the non-numerical data from the data set completely. Another limitation identified during the GUI tests was that when "Names" is selected under "Type of Data" and the user additionally filters for specific names only, all names are pseudonymized regardless of the filtering. The user must carefully select what is to be pseudonymized and what remains unchanged.

5.2 Library Performance Testing

The performance tests focused on the efficiency of the pseudonymization functions, considering the time required to execute each function depending on the input data size and other time-consuming parameters. All tests were conducted with the MacBook Air 2022, Apple M2 chip with 8-core CPU and 10-core GPU, 16 GB unified memory, macOS Ventura 13.2.1, and 512 GB SSD (Macintosh HD). As always, the data was divided into structured and unstruc-

tured categories. Furthermore, it was decided to compare two extreme use cases: one with the minimum possible parameters and the other with the maximum possible parameters. The first use case refers to the test scenario with the simplest configuration, while the second use case includes the most complex configuration. The dataset for the structured data was the same size in both scenarios and contained 100.000 sample rows, which is considered a medium size in data science. In the case of the unstructured data, a sample free text report was used in which 118 of 473 tokens were recognized as PII.

The simplest configuration required only a pseudonymization method and a column or a specific PII entity to be pseudonymized. On the other hand, the performance test with the maximum possible parameters included all columns or PII entities from the input data, and the sensitive information was additionally encrypted in all mapping tables. All pseudonymization methods available in the library were tested in both scenarios.

Figure 5.1 shows three diagrams comparing the minimum and maximum possible parameters using the different library configurations mentioned above. As expected, the runtime for processing structured data increases with the complexity of the configuration. However, the “faker" method, which uses the Faker library to generate false names, is inefficient for large amounts of data, as it takes more than twice as long as other methods, even for the simplest configurations. The runtimes for unstructured data are significantly longer than for structured data, which is due to the fact that the PII must first be recognized and only then pseudonymized.

In addition, the performance was measured with various volumes of structured datasets, as shown in the third line graph in Figure 5.1. The configuration with the shortest runtime from the first chart ("counter") was selected as the minimum possible configuration, and the configuration with the longest runtime ("merkle-tree") was selected as the maximum possible configuration. Unstructured data processing was also tested on a more extensive dataset. However, runtimes remained consistent and reasonable as the amount of PII, even for large texts, is generally lower than for structured data, where each value in the line must be processed. For example, Wikipedia’s biography of Elon Musk contains around 54000 tokens, including the bibliography, but it only amounts to ca.2040 named entities, such as names, places, and organizations. As a result, it took nearly $22,5$ seconds to pseudonymize them all using the “faker” method.

Overall, the library handles data pseudonymization relatively efficiently, although there is room for optimization, especially for methods such as "faker", which has higher runtime.

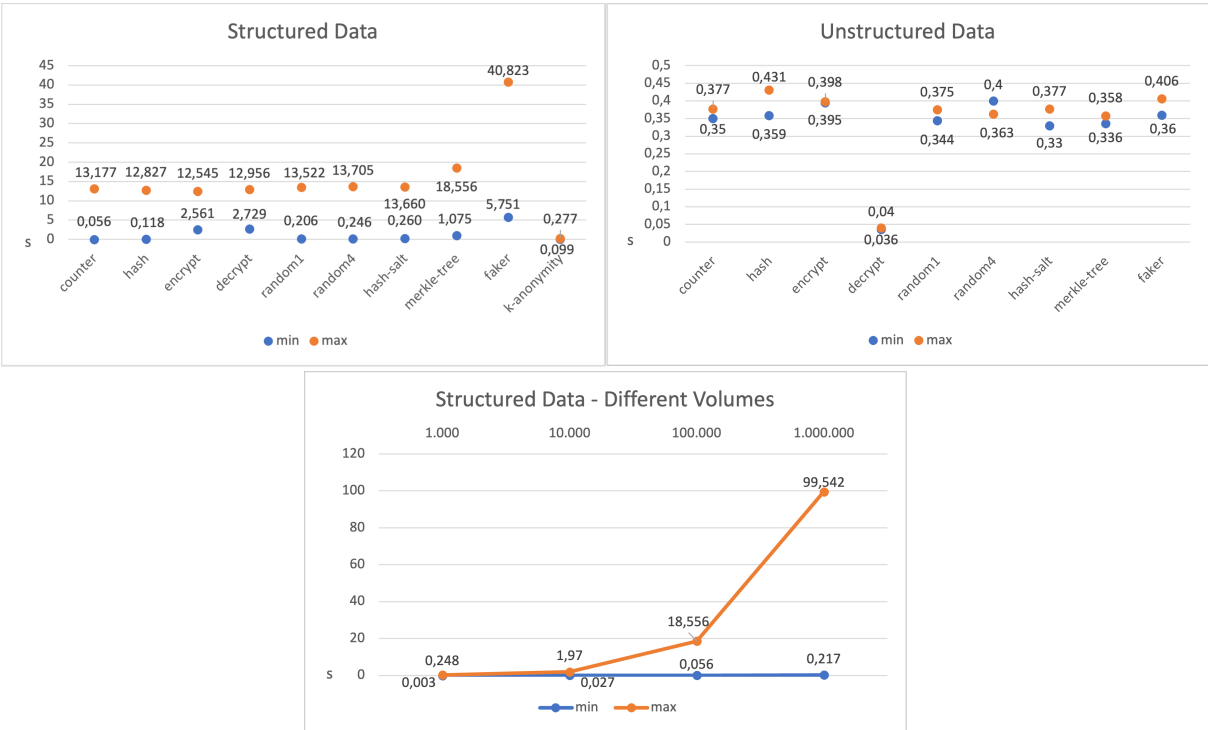


Figure 5.1: 1.Structured, runtime comparison between configurations, 100.000 input rows with five columns, time in seconds.
2. Unstructured, runtime comparison between configurations, 473 input tokens, 118 PII, time in seconds.
3. Structured, runtime comparison between different data volumes and configurations, time in seconds.

5.3 k-Anonymity Testing

The k-anonymization was tested using the $is_k_anonymized()$ function, which checks whether the dataset satisfies k-anonymity. This subsection shows some results on how the data can be minimized and de-identified while still retaining the original general features and thus being useful for further applications.

As an example, consider a plain structured dataset as input. The dataset consists of five columns, only one of which, "Salary", is numerical and in this case is k-anonymized. The other columns are removed. The first bar chart in Figure 5.2 shows what the "Salary" column looks like before anonymization. The result in the second diagram is obtained by applying $k = 3$. Unfortunately, most of the data was lost and only 116 of 1000 rows were returned. For instance, in Table 5.2, only 17 entries remain for salaries in the 25.000 - 50.000 range, which comprised 120 entries before anonymization. This is a common problem, and performing k-anonymization on larger datasets is no guarantee of less data loss. However, when comparing two graphs, the

data characteristics are preserved. For example, the fluctuations between 50.000 and 120.000 have a similar character, increasing slightly at 80.000 and then decreasing until 120.000. The $is_k_anonymized()$ function also confirms that the dataset is 3-anonymous by returning the value $True$. This test can be reproduced in the unit tests within the library. Detailed input/output examples can be found in section 4.4.5, where the use case of k-anonymization of salaries is also described.

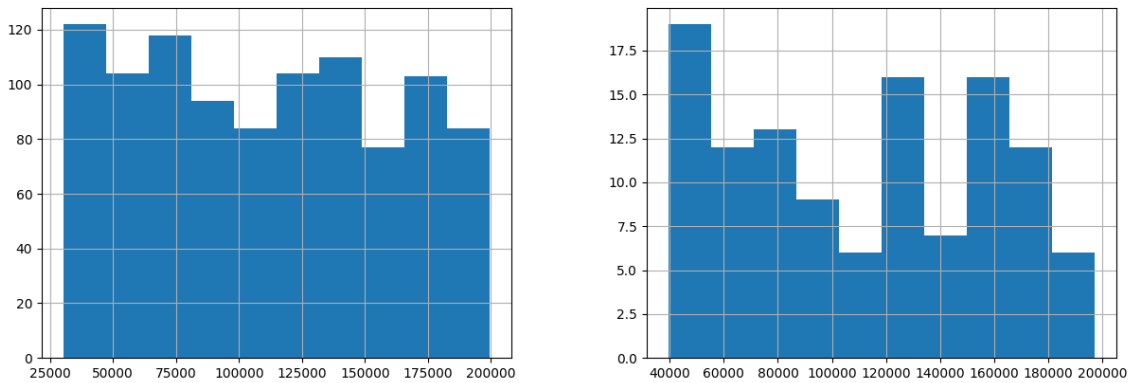


Figure 5.2: Salary data before and after k=3 anonymization (x - salary, y - number of rows)

5.4 Pandas vs. Polars Performance Testing

Although the Polars' syntax differs significantly from Pandas, and the migration can be time-consuming and challenging, Polars' performance benefits are worth the effort. The dedicated unit tests were performed to demonstrate Polars' efficiency in a real-life example. The results are shown in Table 5.1. The tests were conducted with the MacBook Air 2022, Apple M2 chip with 8-core CPU and 10-core GPU, 16 GB unified memory, macOS Ventura 13.2.1, and 512 GB SSD (Macintosh HD). All files listed in the table are located in the folder $pseudPy/test_files/$ in the library repository. The table describes the configurations for each test and displays the results for Pandas' and Polars' versions of library implementations. For the first configuration, the input file consisted of 1.000 rows; one column was selected for hashing, $encrypt_map$ was set to $True$, and, finally, decryption was called. The results showed that Pandas took about 62 seconds to complete the test, while Polars took only $0,14$ seconds for the same configuration. The unit test, described in Performance Testing 5.2, which processes a data set of 100.000 rows using the simplest configuration of the "counter" method, only takes around $0,038$ seconds with Polars, but around $0,310$ seconds with Pandas. In addition, the pseudonymization method "random1" was applied to five different columns. Using Pandas, this process took $1,641$

seconds. In comparison, using Polars, it took almost half the time, completing in $0,791$ seconds.

	Unit Test	Pandas, s	Polars, s
1.	"Worst Case Configuration" File name: plain_user_data.csv Number of rows: 1.000 Column: name Method: hash + decrypt at the end Mapping encryption: True	62	0,14
2.	"Simplest Configuration" File name: MOCK_DATA.csv Number of rows: 100.000 Column: name Method: counter	0.310	0,038
3.	File name: MOCK_DATA.csv Number of rows: 100.000 Columns: name, country, gender, salary, job_title Method: random1	1,641	0,791

Table 5.1: Testing: Pandas vs. Polars

5.5 GUI Evaluation Survey

In order to understand the relevance and find weaknesses in the user interface, an evaluation survey was conducted, which included a demonstration of the software and a final survey. The survey results can be found in Appendix 2. The survey covers topics such as the personal use of the tool by the participants, comprehensibility, user-friendliness, points for improvement and general satisfaction, etc. A total of three people from the areas of risk management in the financial industry, IT consulting and data analysis provided their feedback. All participants chose GUI as a better form of working with pseudonymization than the library and script options. The highest marks were given for the usefulness of the software for their professional background. Potential use cases include analyzing sensitive company financial data that is marked as strictly

confidential. One interviewee mentioned that it is common in their company for employees to be asked to anonymize data by manually replacing sensitive information with a token "x", which usually takes a lot of time and can be automated with the pseudonymization tool. Another point was that the tool is secure to insert the sensitive data, as the data is not stored anywhere outside the company, unlike the web applications that can also serve as pseudonymization tools, such as ChatGPT, and may not be reliable in terms of storing sensitive data. It was also mentioned that in the consulting industry, where the client entrusts the company with confidential data, the use of unreliable tools can lead to a loss of trust, the business relationship and financial consequences.

All participants stated that the usage and functionalities of the graphical user interface are intuitive as long as the documentation or demonstration explains all functions prior to use. One of the comments was that it would be useful to add the pop-up information when the user hovers over the parameter, giving some tips on how to use the specific parameter. Another issue was that browsing file paths always starts with the root directory, such as "Macintosh HD", which is inconvenient because a large organization with typically long folder paths has to manually choose the direction of the folder each time, which could be a pain point for a user. It would therefore be better if browsing the path always started with the last folder visited. It was also suggested that the file reverted to the original output file should also contain the pseudonyms along with the real names, so that if the user needs to uncover a large number of pseudonyms, they will know which pseudonyms correspond to which real names. In addition, the improvement of the app's UX/UI with a more user-friendly interface was recommended. Finally, the question was asked whether the idea of the tool seemed original. Both interviewees stated that they were not aware of any similar tool, even though they work for fairly large companies. Overall, participants were satisfied with the tool's functions while the survey provided further ideas for future improvements.

6 Discussion

Most of the literature sources reviewed in this thesis, Chapter 2, show that the problem of data protection, including pseudonymization, is a critical challenge and that more concrete solutions need to be introduced. With the amount of personal data stored by companies on a daily basis and the numerous data leaks that regularly appear in the news, it is more important than ever to address this issue and offer a possible solution. This chapter discusses the thesis results and mentions the limitations and difficulties of implementing the automatic pseudonymization library. In its early stages, the project was divided into three main phases: investigation of current practices, implementation, and testing. First, the literature was reviewed to determine the best practices and state-of-the-art solutions for the problem. The solution was then implemented as a functionality within the library. Finally, it was tested with unit and other appropriate tests, Chapter 5. An attempt was made to pull every idea through this cycle. So, once the previous idea had been tested, all three phases were reiterated.

For the literature research, relatively diverse sources of information were collected, considering different opinions, critiques, and approaches to pseudonymization. It was also essential to find the most recent works that consider the application of the latest technologies, such as AI or NLP. Another point concerning the literature review is that most papers on pseudonymization deal with theoretical rather than practical aspects and focus on medical applications rather than general applicability. Furthermore, the current official regulations are vague and do not provide concrete examples of handling pseudonymized data, leaving more room for individual interpretation, which can sometimes be confusing and insecure if misused.

The methods and approaches chosen for the library's design and implementation were appropriate regarding efficiency and effectiveness. When using the library, the user can not only pseudonymize or anonymize the data but also learn about data protection methods and measures. Although the library and GUI are designed for automatic pseudonymization, the critical assessment of the situation and the subsequent configuration of input arguments and secure data storage is entirely the responsibility of the user. So, before using any of the available meth-

ods, it is recommended that users read this work and familiarize themselves with the library documentation to get optimal results. The library now includes methods for creating pseudonyms such as counter, randomization, hashing, hashing with salts, Merkle tree, encryption, and fake data generation. In addition, k-anonymity can be a solid measure for de-identifying and generalizing single personal records in the dataset. The results of the performance tests, which are described in detail in Chapter 5, show that the processing time is still manageable and affordable even with large data volumes of million rows. Beyond this, the performance tests have shown that the library is efficient compared to other solutions in the field of data de-identification, such as Presidio. The main goals of highlighting the need for privacy protection, collecting and applying current practices in pseudonymizing data, and developing a convenient and practical tool for a broad user group were achieved. The selected pseudonymization and anonymization techniques combine current practices and personal visions. Most methods have been designed to be irreversible and theoretically cannot be compromised by attackers, although this has not yet been tested and remains to be done in the future. This applies in particular to hashing with SHA-256, counter method, AES encryption, etc. It is intended that the attacker or the data controller can only uncover the identity behind the pseudonym with the help of additional information stored separately and in a secure location. This complies with the GDPR regulations. The key features and functionalities defined in the concept have been implemented. The graphical user interface was also developed as part of the project. It is a perfect example of how to apply a library and is also practical for users without programming experience.

Accelerating the library's runtimes was one of the most time-consuming processes during implementation, involving the migration from Pandas to Polars. Such a migration is seen as a challenge, as Pandas and Polars have different APIs, and therefore, most of their structures, functions and syntax differ. The migration was error-prone and required a lot of debugging. The Polars package is still gaining popularity, so not many alternative sources provide more complex examples. Also, some Pandas functions are unavailable in Polars, so even more lines of code and problem-solving were involved. Experiments were also carried out with various forms of output. It had to be decided whether the input and output data were user-friendly and easy to interpret. The unit tests, by contrast, helped to ensure the robustness of the functionality and to eliminate the major bugs. Another process that required a lot of effort during the implementation was the reorganization of the code, as this was an important point to make the code adaptable and to facilitate the configuration of the new custom pseudonymization methods. Therefore, it was decided to collect all pseudonymization methods in the corresponding dictionary and call the required function based on the name of the chosen method. In addition, similar functionalities were categorized by class and the utility functions were defined for general use, without hard-coded conditions for some specific pseudonymization methods.

The implementation aimed to build an adaptable, fast, and easy-to-use library. As expected, all points were achieved to a certain extent. However, there is always room for improvement and adjustments. The pseudonymization of unstructured data is slower than with structured data. This is due to the fact that NLP must first recognize the personal data. There might, however, be a way to improve performance, which will have to be investigated in the future. The efficiency of the "faker" pseudonymization method must also be reviewed, as the technique is highly time-consuming for bigger data sets, even with structured data. This could be due to the fact that the Faker generates high-quality fake data, which may not be optimized for high-performance scenarios with large amounts of data. However, no specific information on the performance of the Faker package was found. This can also be addressed in future work. The major limitation in the implementation was the time factor, as, indeed, not all bugs could be found in the short testing time. Apart from the techniques defined in the implementation, many other existing pseudonymization techniques could also be used in the library, which are more complex and require more review, testing, and understanding capacity. Another limitation was the lack of knowledge of technologies such as Polars, tkinter, spaCy, and Faker. This meant that a certain amount of learning time was required before they could be deployed. The interface created with tkinter could also have a better design and visual appearance, which can be improved in the future. However, all of the mentioned packages are well-documented, so experience of working with them was gained relatively quickly. On the other hand, the popularity of Polars is still emerging, and mostly only official documentation was the primary source of knowledge. Overall, the large Python community provides excellent help even for Python beginners with previous knowledge of other programming languages. Further points of improvement, which are also mentioned in Chapter 5, are the limited data filtering for structured data with only one condition per initialization, the k-anonymization and aggregation only for numerical data, etc. In the case of unstructured data, the entity recognition engine is not always a guarantee that all personal data will be found. In particular, the unusual entities that do not really match the predefined regex patterns can be overlooked.

The survey results regarding the graphical user interface showed that the software is helpful for people with different professional backgrounds. It is intuitive as long as the documentation or demonstration explains all functions before use, and their companies currently do not utilize analogs. The survey also provided valuable suggestions for improvement, such as the app's UX/UI with a more user-friendly interface, recommendations on the pop-up instructions for each function and better folder navigation, which could also be implemented in the future. It would however be of advantage if a broader group of participants were involved.

To sum up, this project overview highlighted the successes, indicated limitations, and addressed the difficulties encountered during the implementation. Multiple relevant methods for data pseudonymization could be integrated and simultaneously made to work automatically, efficiently, and effectively. Although the library has weak points, future improvements can make it even more robust and bring it closer to perfection.

7 Conclusion

The GDPR states that sensitive personal data must be stored securely and pseudonymized if such data is required for further storage and processing. This includes the names, ID numbers, and other possible demographic data, such as gender, location, and cultural or social identities. The scope of PII goes beyond the data of a specific person, as confidential company information must also be protected from re-identification. Pseudonymization must ensure that no identity can be restored without additional information. The additional information allows the data controller to reveal specific data, which can be achieved through decryption or mapping tables. Although both options have advantages and weaknesses, the data controller must protect the pseudonymization secrets, regardless of whether the mapping table or secret keys are involved. This work aimed to identify current practices, discover a state-of-the-art solution to the problems of pseudonymization and data protection, and implement this in a convenient and fast automated tool. Several software solutions enable data de-identification, such as Presidio for anonymizing unstructured data or ERDRI.spider for structured medical data. On the other hand, the automatic pseudonymization library offers the advantage of a free, generally usable solution that does not require complex configuration.

There are various approaches to the pseudonymization of unstructured text, ranging from NLP recognition, tokenization, and LLM use to Seq2Seq solutions. The library implementation only considers one of the approaches, NLP, with the combination of Regex for data recognition. The recognized data is later replaced by the pseudonyms generated by the selected pseudonymization method. Six methods for pseudonym generation were developed for both structured and unstructured text data, such as counter, randomization, cryptographic hash function, encryption, Merkle tree or so-called user-generated pseudonyms, faker, and two regarding data minimization, k-anonymity, and aggregation. All methods are designed to be irreversible unless there is any additional information in the form of pseudonymization secrets, such as a secret key for decryption or a mapping table for the other methods. Another key point is that the library has been designed for general and customizable purposes, allowing users to add a custom pseudonymization method. An automatic script that can be executed with the config-

uration YAML file via the terminal has been introduced. Furthermore, the pseudonymization tool has been integrated into the GUI, which enables even better configuration options and user orientation. The test results showed that the library is efficient in terms of runtime performance and effective in terms of its functionalities. The main objective was to simplify the task of pseudonymizing data and make it accessible to a wider range of users. The choice of Python in this situation makes the library more relevant to a large Python community, especially to researchers, data scientists, and any organization that stores sensitive information. Despite challenges such as optimizing the library's runtime, learning and integrating new technologies, and designing simple user interactions and output delivery, the implementation goals were successfully achieved.

For future work, it would be beneficial to include more complex pseudonymization structures and functionalities, like differential privacy or zero-knowledge proof, in the library. The GUI should be redesigned to become more modern, user-friendly, and enhanced with more intuitive features. The runtimes of the "faker" method and the pseudonymization of unstructured data, in general, should also be checked and optimized. Also, k-anonymization can be improved with a better algorithm, and aggregation of non-numeric data can be added. The points for improvement mentioned by the survey participants must also be taken into consideration when optimizing the user interface. Finally, more test cases should be introduced, and the publication of new literature should be monitored to update the library with the latest practice constantly.

List of Tables and Figures

List of Tables

4.1 Possible Output Files 23
4.2 Pseudonymization Methods. 30

5.1 Testing: Pandas vs. Polars 46

List of Figures

2.1 Reversible data example: data linkage 6
2.2 Data-tier separation: examples of pseudonymized datasets 6
2.3 Pseudonymization workflow of emails (German original (left), English translation (right)) 9
2.4 Merkle Tree: pseudonym proving 12

3.1 The concept of the pseudonymization class for structured data. 17
3.2 GUI Mockup for free text page. 19

4.1 Left: Page for Structured Data; Right: Page for Unstructured Data 32
4.2 Left: revert the data; Right: k-anonymize the data 32
4.3 Configuration Example - Use Case 4 36

4.4 Flowchart: Program Flow of Data Pseudonymization. 38
4.5 Flowchart: Program Flow of Data Aggregation. 39
4.6 Flowchart: Program Flow of Data k-Anonymization. 40

5.1 1.Structured, runtime comparison between configurations, 100.000 input rows with five columns, time in seconds. 2. Unstructured, runtime comparison between configurations, 473 input tokens, 118 PII, time in seconds. 3. Structured, runtime comparison between different data volumes and configurations, time in seconds. 44
5.2 Salary data before and after k=3 anonymization (x - salary, y - number of rows) . 45

Bibliography

[1] Explosion AI. *spaCy Usage: Facts & Figures*. Accessed: 2024-05-30. 2024. URL: https://spacy.io/usage/facts-figures.

[2] Massimo Bertaccini. *Cryptography Algorithms: A Guide to Algorithms in Blockchain, Quantum Cryptography, Zero-Knowledge Protocols, and Homomorphic Encryption*. Birmingham: Packt Publishing, Limited, 2022. ISBN: 9781789618570.

[3] Iain Bourne. "Personal, Pseudonymous, and Anonymous Data: The problem of identification". In: *XRDS* 20.1 (2013), 27–31. ISSN: 1528-4972. DOI: 10.1145/2508972. URL: https://doi.org/10.1145/2508972.

[4] Joint Research Centre. *SPIDER: An EU-wide platform for collaborative research on public health*. https://eu-rd-platform.jrc.ec.europa.eu/spider/. Accessed: 2024-05-27. 2024.

[5] European Union Agency for Cybersecurity (ENISA). *Deploying Pseudonymisation Techniques*. https://www.enisa.europa.eu/publications/deploying-pseudonymisation-techniques. Accessed: 2024-05-07. (2022).

[6] European Union Agency for Cybersecurity (ENISA). *Pseudonymisation techniques and best practices*. https://www.enisa.europa.eu/publications/pseudonymisation-techniques-and-best-practices. Accessed: 2024-05-05. (2019).

[7] Elisabeth Eder et al. ""Beste Grüße, Maria Meyer" — Pseudonymization of Privacy-Sensitive Information in Emails". In: *Proceedings of the Thirteenth Language Resources and Evaluation Conference*. Ed. by Nicoletta Calzolari et al. Marseille, France: European Language Resources Association, June 2022, pp. 741–752. URL: https://aclanthology.org/2022.lrec-1.79.

[8] International Organization for Standardization (ISO). *Health informatics - Pseudonymization*. ISO/TS 25237:2008(E). International Organization for Standardization, 2008.

[9] Farhana Jabeen et al. "Enhanced Architecture for Privacy Preserving Data Integration in a Medical Research Environment". In: *IEEE Access* 5 (2017), pp. 13308–13326. DOI: 10.1109/ACCESS.2017.2707584.

[10] Georgios Kermezis, Konstantinos Limniotis, and Nicholas Kolokotronis. "User-generated pseudonyms through Merkle trees". In: *CoRR* abs/2109.03673 (2021). arXiv: 2109.03673. URL: https://arxiv.org/abs/2109.03673.

[11] R. Lautenschläger. *Pseudonymization in Biomedical Research*. https://mediatum.ub.tum.de/doc/1445854/document.pdf. Accessed: 2024-05-07. (2018).

[12] R. Lautenschläger et al. "A generic solution for web-based management of pseudonymized data". In: *BMC Medical Informatics and Decision Making* 15 (2015), p. 100. DOI: 10.1186/s12911-015-0222-y. URL: https://doi.org/10.1186/s12911-015-0222-y.

[13] Alfred J. Menezes, Paul C. Van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. Boca Raton [u.a.]: CRC Press, 1997, pp. 33, 321–322. ISBN: 0849385237.

[14] Adam Meyerson and Ryan Williams. "On the complexity of optimal K-anonymity". In: *Proceedings of the Twenty-Third ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. PODS '04. Paris, France: Association for Computing Machinery, 2004, 223–228. ISBN: 158113858X. DOI: 10.1145/1055558.1055591. URL: https://doi.org/10.1145/1055558.1055591.

[15] Microsoft. *Presidio*. https://github.com/microsoft/presidio. Accessed: 2024-05-27. 2024.

[16] Zhao-Yan Ming and Tat Seng Chua. "Resolving Polysemy and Pseudonymity in Entity Linking with Comprehensive Name and Context Modeling". In: *Information Sciences* 307 (2015), pp. 18–38. ISSN: 0020-0255. DOI: 10.1016/j.ins.2015.02.025. URL: https://www.sciencedirect.com/science/article/pii/S0020025515001255.

[17] Joseph P. Near and Chiké Abuah. *Programming Differential Privacy*. Vol. 1. Accessed: 2024-05-30. 2021. URL: https://programming-dp.com/.

[18] J. Neerbek. "Sensitive Information Detection: Recursive Neural Networks for Encoding Context". In: *arXiv* (2020). Accessed: 2024-05-06.

[19] R. Noumeir, A. Lemay, and J.-M. Lina. "Pseudonymization of Radiology Data for Research Purposes". In: *Journal of Digital Imaging* 20.3 (2007), pp. 284–295. DOI: 10.1007/s10278-006-1051-4.

[20] The European Parliament and The Council of The European Union. "Regulation (EU) 2018/1725 of the European Parliament and of the Council of 23 October 2018". In: *Official Journal of the European Union* (2018). Accessed: 2024-05-06.

[21] K. Pommerening et al. *Leitfaden zum Datenschutz in medizinischen Forschungsprojekten*. 1st ed. Berlin: MWV, 2014. ISBN: 3954661233.

[22] Chaudhary Priyanshu. *Pandas 2.0 vs. Polars: The Ultimate Battle*. Accessed: 2024-05-30. 2023. URL: https://medium.com/cuenex/pandas-2-0-vs-polars-the-ultimate-battle-a378eb75d6d1.

[23] Arun Amaithi Rajan and Anitha Amaithi Rajan. “Data Anonymization Techniques for Preserving Privacy in Public Release Data Model: A Technical Review”. In: *International Journal of Scientific Research in Computer Science and Engineering* 8 (1 2020), pp. 58–62. ISSN: 2320-7639. DOI: 10.26438/ijsrcse/v8i1.5862.

[24] L. Rocher, J.M. Hendrickx, and YA de Montjoye. “Estimating the success of re-identifications in incomplete datasets using generative models”. In: *Nat Commun* 10.3069 (2019).

[25] Mike Rosulek. “Chapter 11: Hash Functions”. In: *The Joy of Cryptography*. 2021, pp. 204–205. URL: https://joyofcryptography.com/pdf/chap11.pdf.

[26] Pierangela Samarati and Latanya Sweeney. “Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression”. In: 1998. URL: https://api.semanticscholar.org/CorpusID:2181340.

[27] S. Y. Shin and H. S. Kim. “Data Pseudonymization in a Range That Does Not Affect Data Quality: Correlation with the Degree of Participation of Clinicians”. In: *Journal of Korean Medical Science* 36.44 (Nov. 15, 2021), e299. DOI: 10.3346/jkms.2021.36.e299.

[28] Statista. *Annual number of data compromises and individuals impacted in the United States from 2005 to 2023*. Accessed: 2024-06-12. 2024. URL: https://www.statista.com/statistics/273550/data-breaches-recorded-in-the-united-states-by-number-of-breaches-and-records-exposed/.

[29] Oleksandr Yermilov, Vipul Raheja, and Artem Chernodub. *Privacy- and Utility-Preserving NLP with Anonymized Data: A case study of Pseudonymization*. 2023. arXiv: 2306.05561 [cs.CL].

Appendices

Appendix 1: Library Documentation

Welcome to pseudPy's documentation!

Modules

class `Pseudonymization.`**`Aggregation`**(`column`, `method`, `df=None`, `input_file=None`, `output=None`)

Bases: **`object`**

Class for data aggregation.

Parameters:

- **column** (*str*) – Column to aggregate.
- **method** (*list*) – Aggregation method and the range - ['number', int] for numerical data or ['dates-to-years', int] for dates.
- **df** (*Pandas Dataframe*) – Input Dataframe. Required if input path is not provided.
- **input_file** (*str*) – Path to input file. Required if dataframe is not provided.
- **output** (*str*) – Path to output folder.

`group`()

Main function to initiate aggregation of csv.

Return type: Output to file, if the output path is passed, or to a Pandas Dataframe.

Example

Aggregate salary data to the groups of range 10000.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> agg = pseudPy.Aggregation(
>>>     column='salary',
>>>     method=['number', 10000],
>>>     input_file='some_user_data.csv',
>>>     output='/output/dir')
>>>
>>> agg.group()
```

class `Pseudonymization.`**`KAnonymity`**(`df`, `k`, `depths=None`, `mask_others=False`, `output=None`)

Bases: **`object`**

Class for data k-anonymization.

Parameters:

- **df** (*Pandas Dataframe*) – Input data.
- **k** (*int*) – The level of anonymity. Each row in the data must be matched to at least k other rows.
- **depths** (*dict*) – The depths of rounding the values in form {column_name: depth}. Ex. value=55222, depths = {value: 1}, result=55220.
- **mask_others** (*bool*) – Enable or disable tokenizing all string values with token ' * '. To achieve k-anonymity, string data must be masked in most cases, unless it has also been k-anonymized externally beforehand.
- **output** (*str*) – Path to output folder.

`is_k_anonymized`()

Check if the data is k-anonymous.

Return type: True or False.

Example

Check if the data is k-anonymous.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> grouped = pl.read_csv('/path/to/input.csv')
>>> is_k_anonym = pseudPy.KAnonymity(
>>>             df=grouped,
>>>             depths={'salary': 1},
>>>             k=2)
>>>
>>> print(is_k_anonym.is_k_anonymized())
```

k_anonymity ()

k-anonymize the data by providing the dataframe, k, depths of anonymization.

Return type: k-anonymized Dataframe or output to file, if the output path is passed.

Example

k-anonymize salary data with k of 2.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> df = pl.read_csv('/path/to/input.csv')
>>> k_anonymity = pseudPy.KAnonymity(
>>>             df=df,
>>>             depths={'salary': 1},
>>>             k=2,
>>>             output='/output/dir')
>>>
>>> grouped = k_anonymity.k_anonymity())
```

class Pseudonymization.**Pseudonymization** (map_method='counter', map_columns=None, input_file=None, output=None, df=None, mapping=True, encrypt_map=False, text=None, all_ne=False, seed=None, pos_type=None, patterns=None)

Bases: **object**

Main class for data pseudonymization.

Parameters:

- **map_method** (*str*) – Pseudonymization method. Select one of the following: *'counter', 'random1', 'random4', 'hash', 'hash-salt', 'merkle-tree', 'encrypt', 'decrypt', 'faker'*. Or specify the faker method: *'faker-name', 'faker-loc','faker-email', 'faker-phone', 'faker-org'*.
- **map_columns** (*str or list*) – Column(s) to be pseudonymized for structured data.
- **input_file** (*str*) – Path to input file. Use if the parameter df is not specified.
- **output** (*str*) – Path to output folder. Use if the output to file is required.
- **df** (*Polars DataFrame*) – Input CSV, read as Polars DataFrame. Use if data is structured and the input_file is not specified.
- **mapping** (*bool*) – Enable or disable mapping output. Required for reversibility of pseudonyms.
- **encrypt_map** (*bool*) – Enable or disable encryption of the mapping table. Output includes additionally secret key file for decryption.
- **text** (*str*) – Input text. Use if data is unstructured and the input_file is not specified.
- **all_ne** (*bool*) – Enable or disable pseudonymization of all named entities such as names, locations, and organizations. Use if data is unstructured and no other entities have to be pseudonymized.
- **seed** (*int*) – Seed random pseudonymization methods like random1, random4, or faker. Return always expected result.
- **pos_type** (*str or list*) – Type(s) of entities in data to be pseudonymized such as 'Names', 'Locations', 'Organizations', 'Emails', and 'Phone-Numbers'. Use if data is unstructured.
- **patterns** (*str or spaCy Matcher*) – If data is structured, use *"column,operation,value"*, else *patterns = [[{"LOWER": "abc"}, {"LOWER": "corporation"}]…]*.

`nlp_pseudonym`()

Main function for pseudonymization of free text.

Returns: *A pseudonymized String or, if output parameter is passed, writes the pseudonymized String and mappings to files. If the encryption is involved, the secret keys are written to the .txt files by default.*

Example

Pseudonymization of unstructured data using 'merkle-tree' method and filtering.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> pseudo = pseudPy.Pseudonymization(
>>>        map_method = 'merkle-tree',
>>>        input_file = '/path/to/input.txt',
>>>        output='/output/dir',
>>>        patterns = [[{"LOWER": "Emily"}, {"LOWER": "White"}],
>>>        [{"LOWER": "Bob"}]])
>>>
>>> pseudo.nlp_pseudonym()
```

`pseudonym`()

Main function for pseudonymization of csv data.

Returns: *Pseudonymized Dataframe or a String. If output parameter is passed, writes pseudonymized and mapping files. If the encryption is involved, the secret keys are written to the .txt files by default.*

Example

Pseudonymization of structured data using 'faker-name' method and filtering.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> pseudo = pseudPy.Pseudonymization(
>>>        map_method = 'faker-name',
>>>        map_columns = 'names',
>>>        input_file= '/path/to/data.csv',
>>>        output='/output/dir',
>>>        patterns=['salary', '>', 100000])
>>>
>>> pseudo.pseudonym()
```

revert_nlp_pseudonym (revert_df, pseudonyms=None)

Revert free text to original.

Parameters:
- **revert_df** (*Polars DataFrame*) – The mapping table.
- **pseudonyms** (*list*) – Filter for exact pseudonyms to revert. Optional.

Return type: A String or, if output parameter is passed, output as a file.

Example

Revert pseudonymized values to the original. For unstructured data.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> with open('/path/to/input.txt', "r") as file:
>>>      text = file.read()
>>> output = '/output/dir'
>>> pseudo = pseudPy.Pseudonymization(
>>>        map_columns = 'Names',   # select from: 'Names', 'Locations',
>>>        text=text,               #'Organizations', 'Emails' and 'Phone-Numbers'.
>>>        output=output)
>>> df_revert = pl.read_csv(f'{output}/mapping_output_Names.csv')
>>>
>>> pseudo.revert_nlp_pseudonym(df_revert)
```

revert_pseudonym (revert_df=None, pseudonyms=None)

Revert structured data to original in form of Dataframe.

Parameters:
- **revert_df** (*Polars DataFrame*) – The mapping table in form of Polars Dataframe.
- **pseudonyms** (*list*) – Filter for exact pseudonyms to revert. Optional.

Return type: A reverted Dataframe and if output parameter is passed, a file with reverted pseudonyms.

Example

Revert pseudonymized values to the original. For structured data.

```
>>> import pseudPy.Pseudonymization as pseudPy
>>> df = pl.read_csv('/path/to/data.csv')
>>> output='/output/dir'
>>> pseudo = pseudPy.Pseudonymization(
>>>        map_columns = 'column1',
>>>        df=df,
>>>        output=output)
>>> df_revert = pl.read_csv(f'{output}/mapping_output_column1.csv')
>>>
>>> pseudo.revert_pseudonym(df_revert)
```

Indices and tables

- **genindex**
- **modindex**

Appendix 2: GUI Evaluation Form and Results

Pseudonymization Tool - Evaluation

3 responses

Publish analytics

What is your (professional) background?

3 responses

Data Analysis

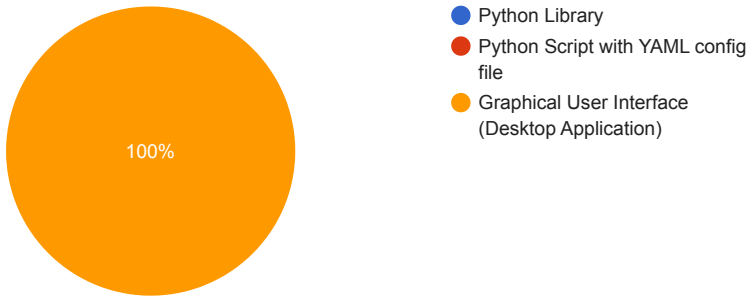
Risk Management: Financial Industry

IT Consulting

Please select the form of pseudonymization software that best suits your needs:

Copy

3 responses



Why would you choose this form?

3 responses

easy in use

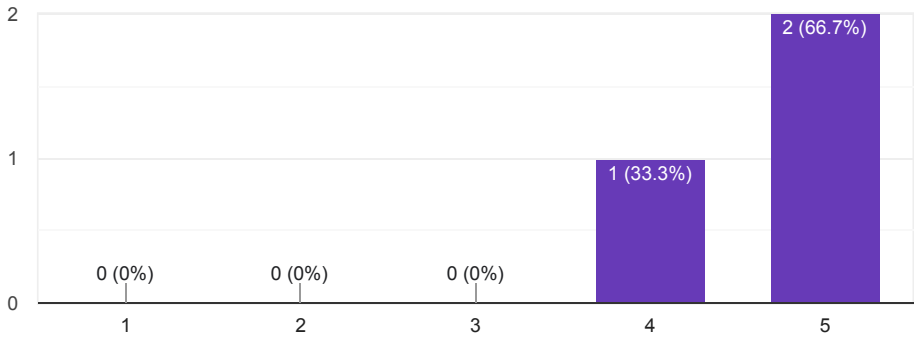
I do not have strong IT skills, therefore in my work I would use the GUI version for my job-related use cases

User-friendly, suites for people without any technical background

In your opinion, how useful would the tool be for your professional background?

Copy

3 responses



In which situations could you imagine using this tool? What problems could it solve?

3 responses

Automatization of the Pseudonymization, Data Protection

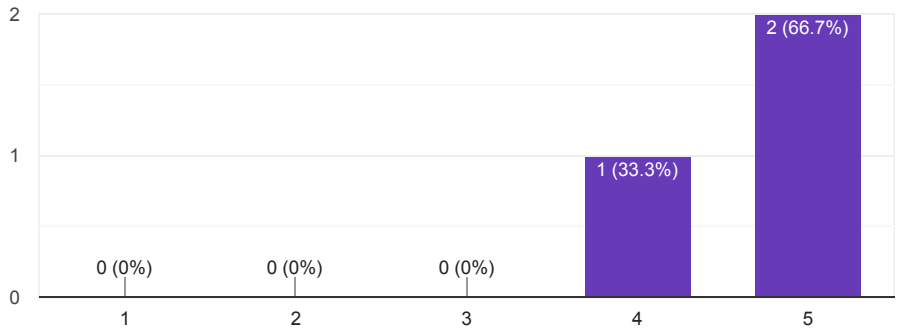
In analysing company-sensitive financial data that is labelled as (strictly) confidential. In order to distribute the results of the analysis, the tool would help me anonymise sensitive information

To anonymize a data of one client to reuse it for other clients. It's important for every client (in consultant industry) to be able to trust its data under consideration that it will not be shown anyone outside of a consultant company. In a case, if real data of one client will be shown others, could lead to loss of trust, business relationship and even monetary consequences.

Understandability of the Pseudonymization GUI

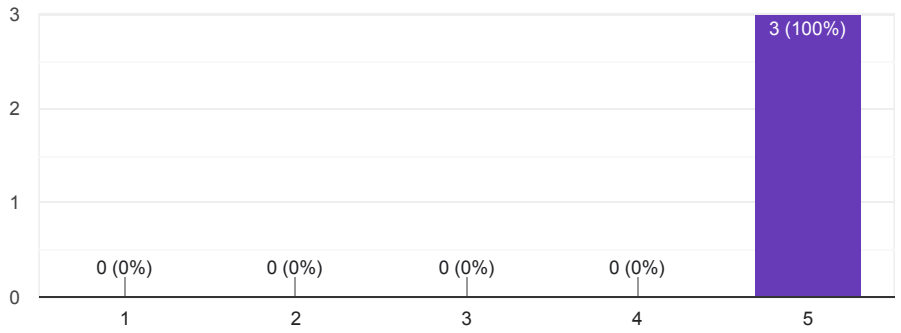
How straightforward is the purpose and the target audience of the tool? Copy

3 responses



How easy are the basic functionalities to use? Copy

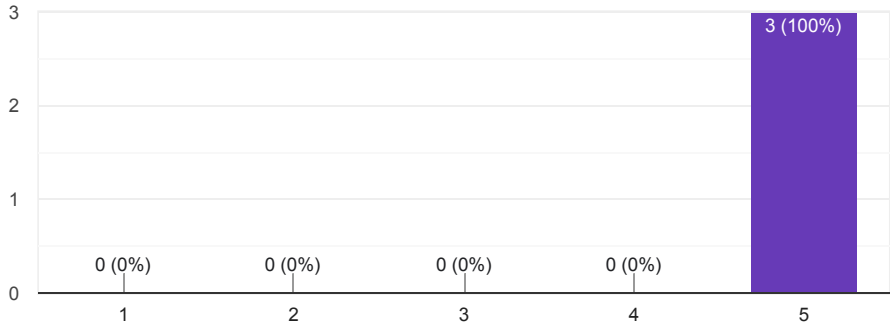
3 responses



How easy are the advanced functionalities to use?

Copy

3 responses

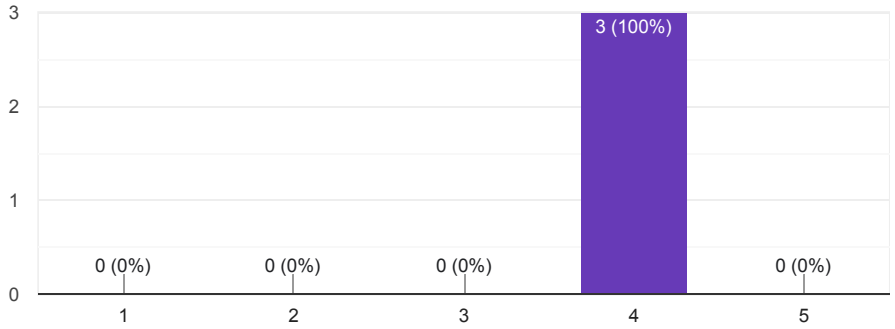


Usability of the Pseudonymization GUI

Copy

Navigation: To what extent is the app's navigation intuitive and easy to use?

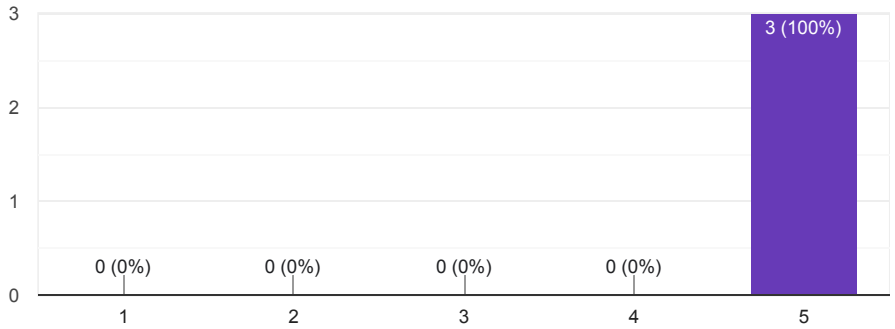
3 responses



Copy

Legibility: To what extent is the text legible and easy to understand? Are the fonts and colors consistent on all pages?

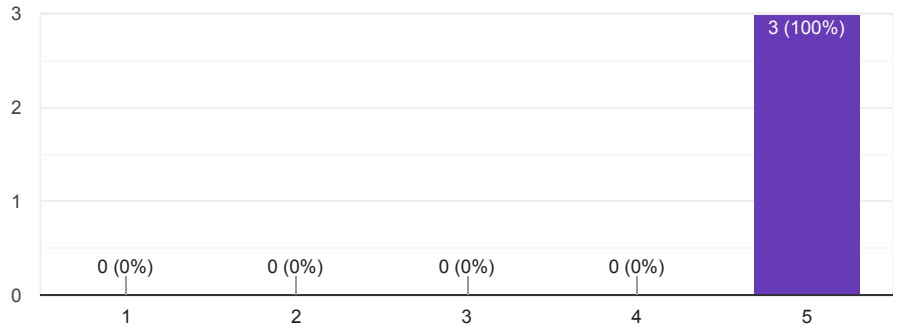
3 responses



Copy

Loading times and speed: In your opinion, are the loading times for pages and data processing acceptable?

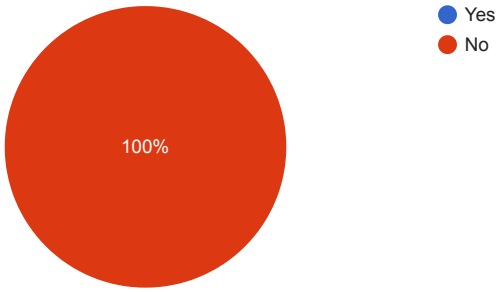
3 responses



Copy

User flow: did you experience frustration during the pseudonymization process?

3 responses



User flow: if yes, at what point of the demo?

0 responses

No responses yet for this question.

More Feedback

Is there anything you would like to change about the software? How would you improve it?

3 responses

In the revert output add the corresponding pseudonym to the real name

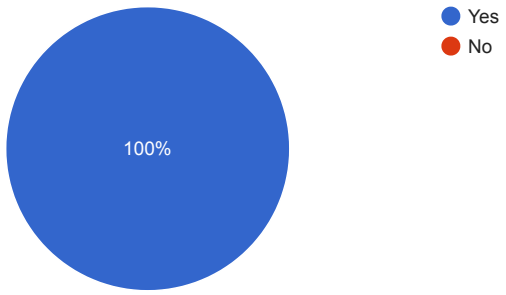
I would improve the GUI sections of Input File Path and Output Folder Path with the saving of the latest folder visited. If the user in a large company with typically long folder paths will have to manually choose the folder direction each and every time, that might be a user's pain point.

Way for improvement: to implement a tooltip for parameters (e.g. Encrypt Mapping) as a purpose of all features is not always clear.

Feature Completeness: Do you feel that the tool has all the necessary features for pseudonymization?

Copy

3 responses



If no, please specify which features you think are missing.

3 responses

Everything is perfect

No pseudonymization features are missing. I would only suggest to improve in the future the UX/UI of the app with an easier user navigation

No

How original is the idea? Do you have any examples of similar apps?

2 responses

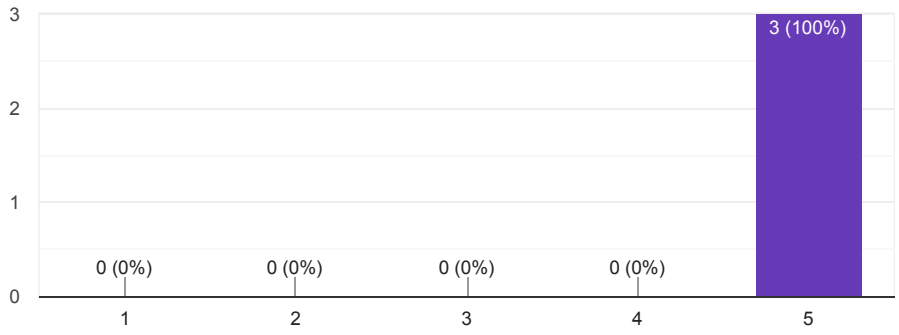
Personally I am not aware of the similar app, but I am sure they might exist. Nevertheless, the tool is really useful.

I didn't have any experience with a similar tool

Overall Satisfaction: How satisfied are you with the overall experience of the pseudonymization tool?

Copy

3 responses



This content is neither created nor endorsed by Google. Report Abuse - Terms of Service - Privacy Policy

Google Forms